ADVANCED ROBOTIC PERCEPTION

MACHINE LEARNING (FOR COMPUTER VISION)
RIKKE GADE 2026



AALBORG UNIVERSITY
DENMARK

Agenda

Today: ”Classic” machine learning methods for computer vision applications

- Unsupervised learning
  - Clustering
- Supervised learning
  - Regression
  - Decision trees
  - Support Vector Machines

Next lecture: Neural networks for computer vision

Many slides from Joseph Redmond, University of Washington, https://pjreddie.com/courses/computer-vision and Andrew Moore, Carnegie Mellon University, https://www.cs.cmu.edu/~./awm/tutorials/index.html



Machine learning

What is machine learning?

Algorithms to approximate functions / find underlying function

- Usually use lots of statistics
- Usually minimize some form of *loss function*
- Learns from training data

Known algorithms from bachelor course:

- k-Nearest Neighbour classifier
- Bayes Classifier



Machine learning

Supervised learning
- Learn from lots of labelled examples
- Reduce the sum of ”costs” (difference between predicted and true label)

Unsupervised learning
- Tries to model unlabelled data
- Find similarities and differences between subgroups of data
- Learn functions to generate new data

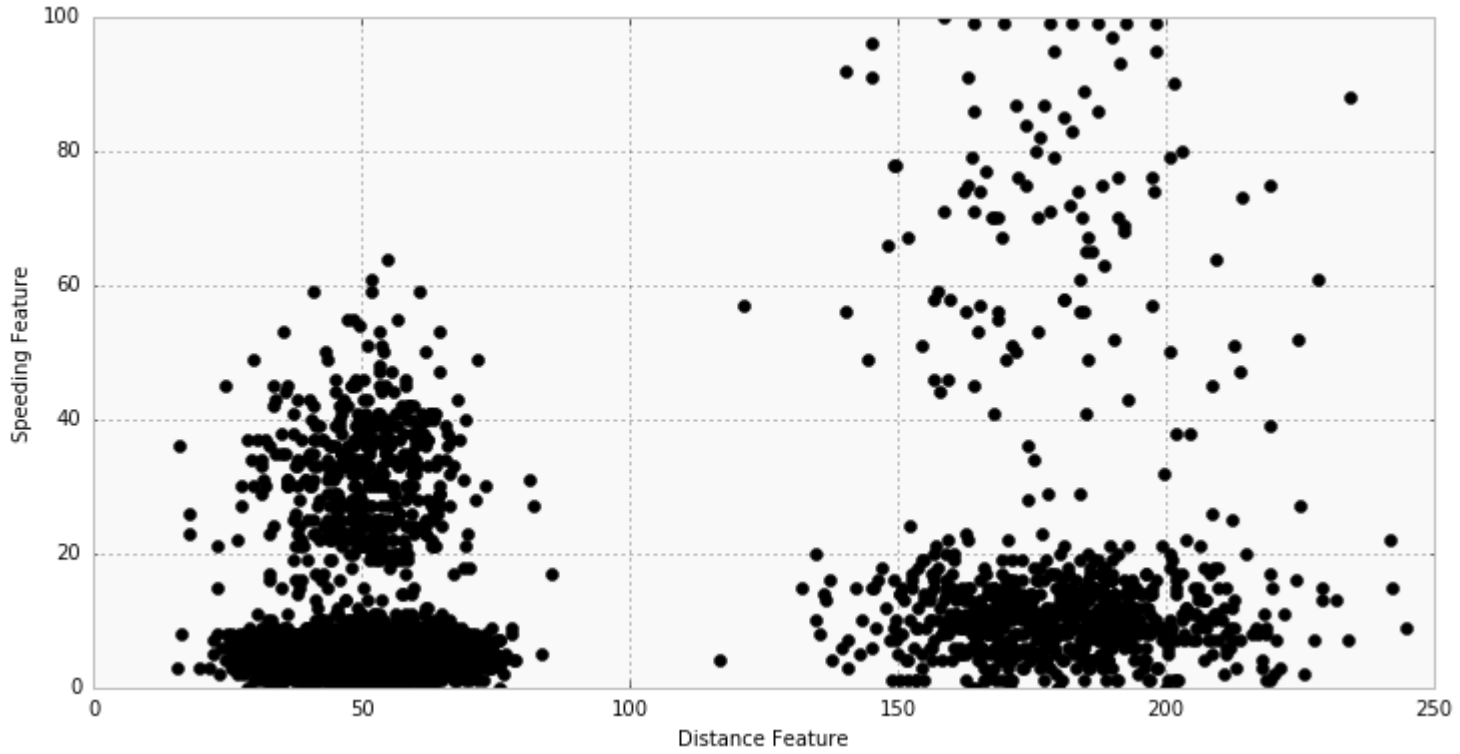
Other categories, like:
Reinforcement learning
Semi-supervised learning
Self-supervised learning
…

Unsupervised learning

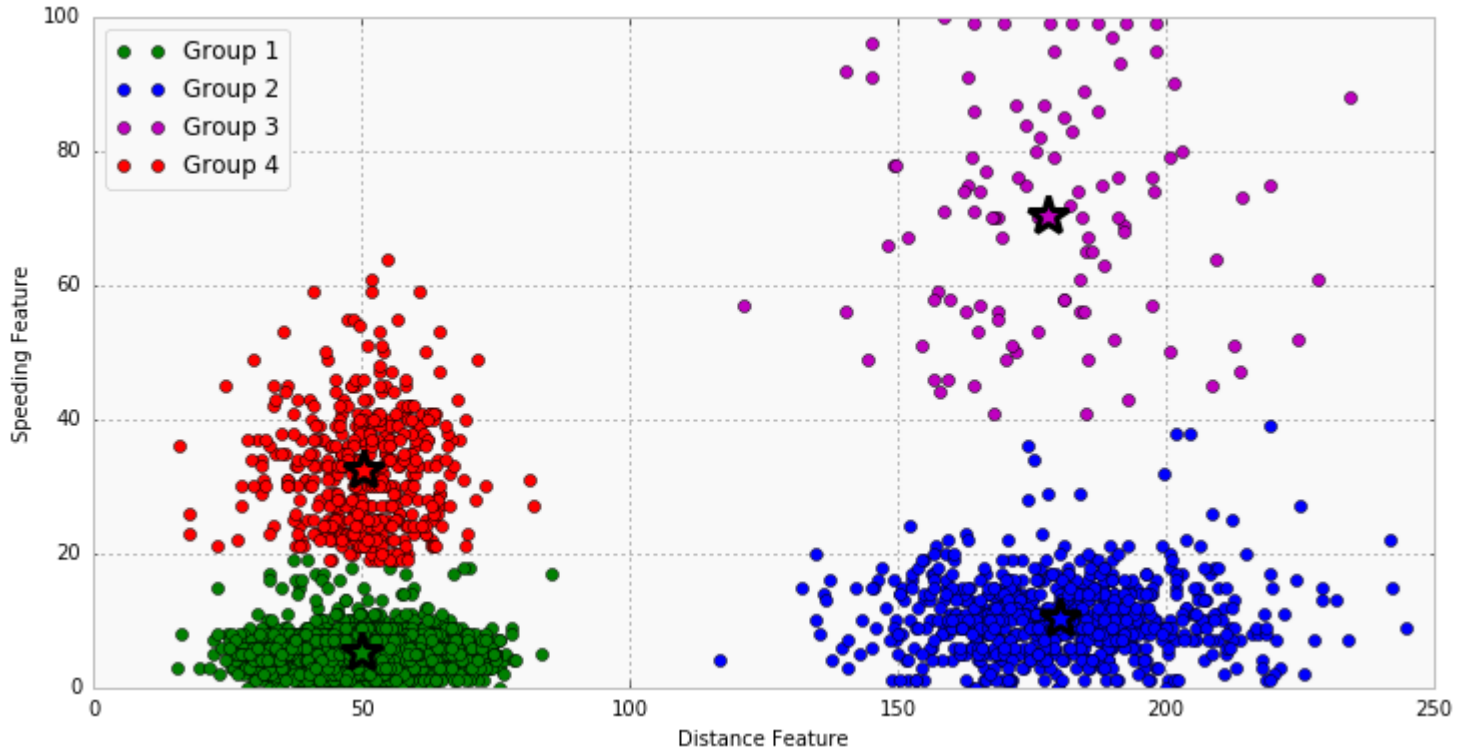
- No labels – have to look for patterns in data
- Clustering: Find groups in data



Clustering


Speeding Feature
Distance Feature
0
20
40
60
80
100
0
50
100
150
200
250

Clustering


Group 1
Group 2
Group 3
Group 4
Speeding Feature
Distance Feature
0
20
40
60
80
100
0
50
100
150
200
250

K-means clustering

Assume points are close to other points in group, far from points out of group

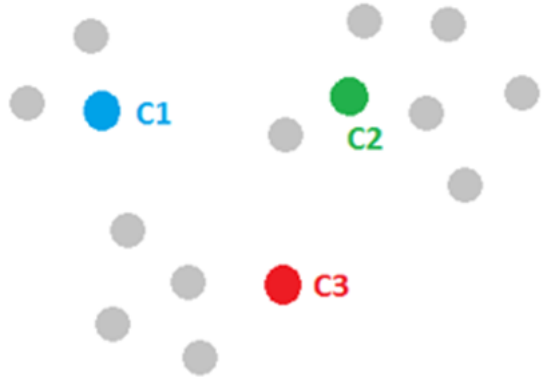


K-means clustering

Assume points are close to other points in group, far from points out of group

Algorithm:

- Randomly initialize *k* cluster centers

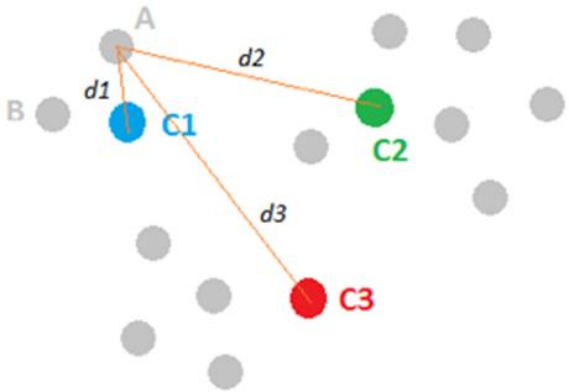


K-means clustering

Assume points are close to other points in group, far from points out of group

Algorithm:

- Randomly initialize *k* cluster centers
- Calculate distance between points and centers

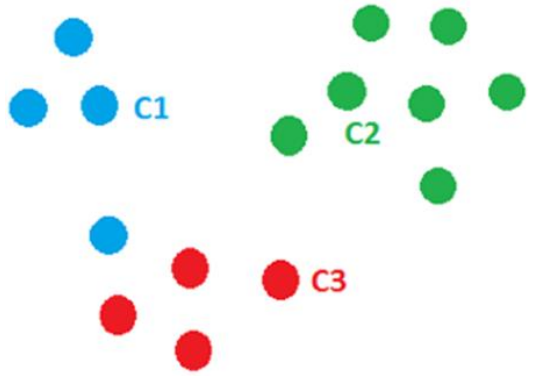


K-means clustering

Assume points are close to other points in group, far from points out of group

Algorithm:

- Randomly initialize *k* cluster centers
- Calculate distance between points and centers
- Assign each point to closest center

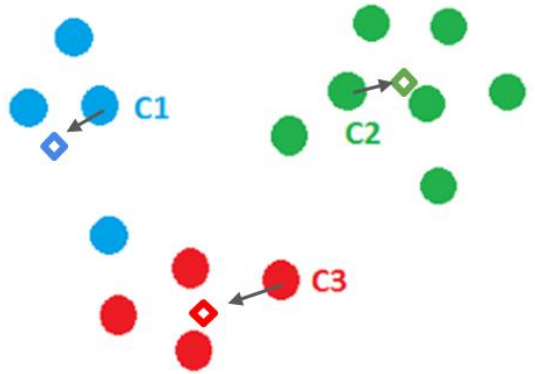


K-means clustering

Assume points are close to other points in group, far from points out of group

Algorithm:

- Randomly initialize *k* cluster centers
- Calculate distance between points and centers
- Assign each point to closest center
- Update cluster centers: average of points

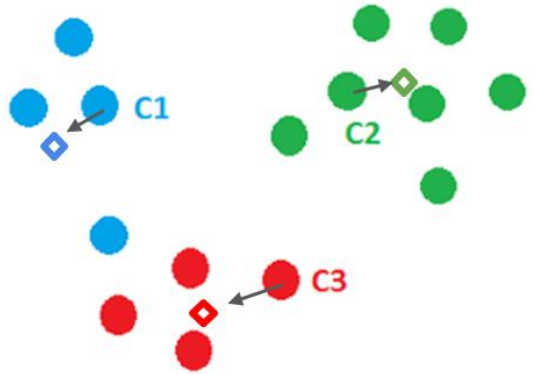


K-means clustering

Assume points are close to other points in group, far from points out of group

Algorithm:

- Randomly initialize *k* cluster centers
- Calculate distance between points and centers
- Assign each point to closest center
- Update cluster centers: average of points
- Repeat

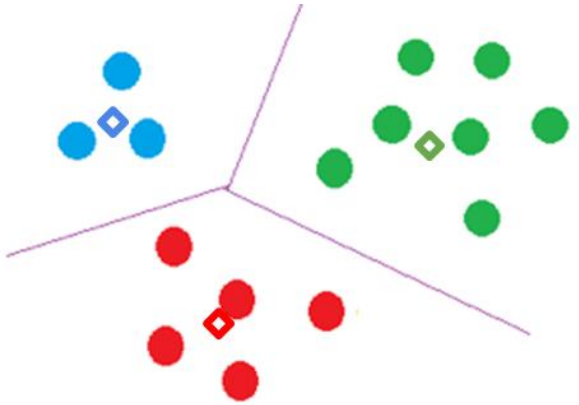


K-means clustering

Assume points are close to other points in group, far from points out of group

Algorithm:

- Randomly initialize *k* cluster centers
- Loop until converged:
  - Calculate distance between points and centers
  - Assign each point to closest center
  - Update cluster centers: average of points



K-means clustering

- Can be used on features, but also on pixel values directly in colour space

- Group together pixels by colour, automatic segmentation
- K-means, k=2





K-means clustering

- Can be used on features, but also on pixel values directly

- Group together pixels by colour, automatic segmentation
- K-means, k=4





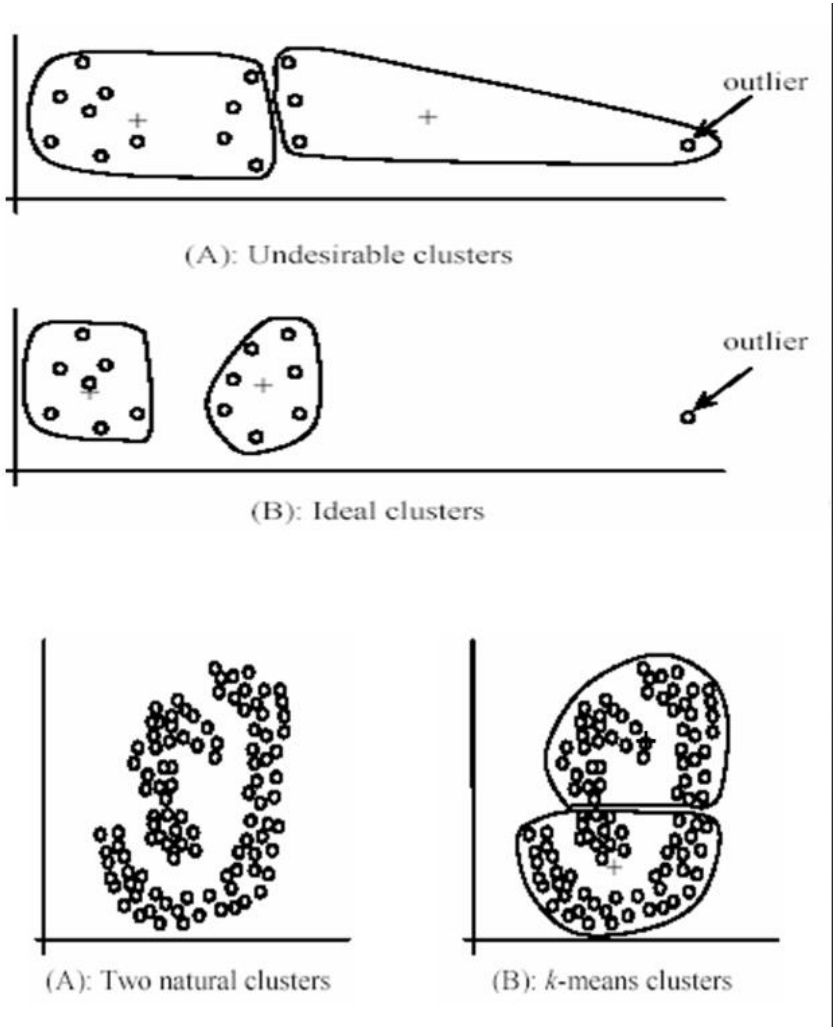
K-means clustering

Pros:

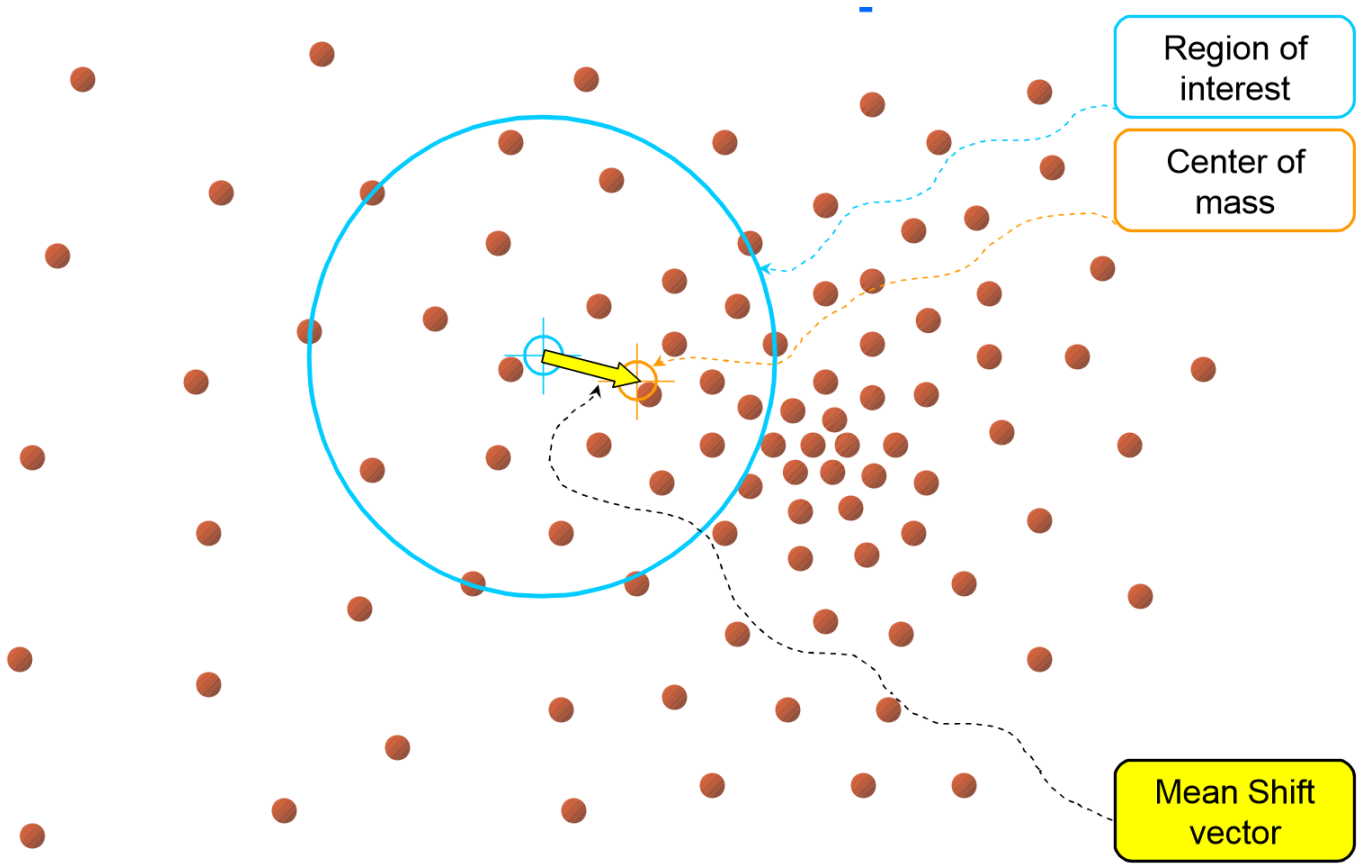
- Simple and fast*
- Easy to implement
- Converges to local minimum of within-cluster squared error

Cons:

- Need to choose K
- Sensitive to outliers
- Sensitive to initial centers
- Prone to local minima
- Detects spherical clusters only
- *Can be slow: each iteration is O(KNd) for N d-dimensional points

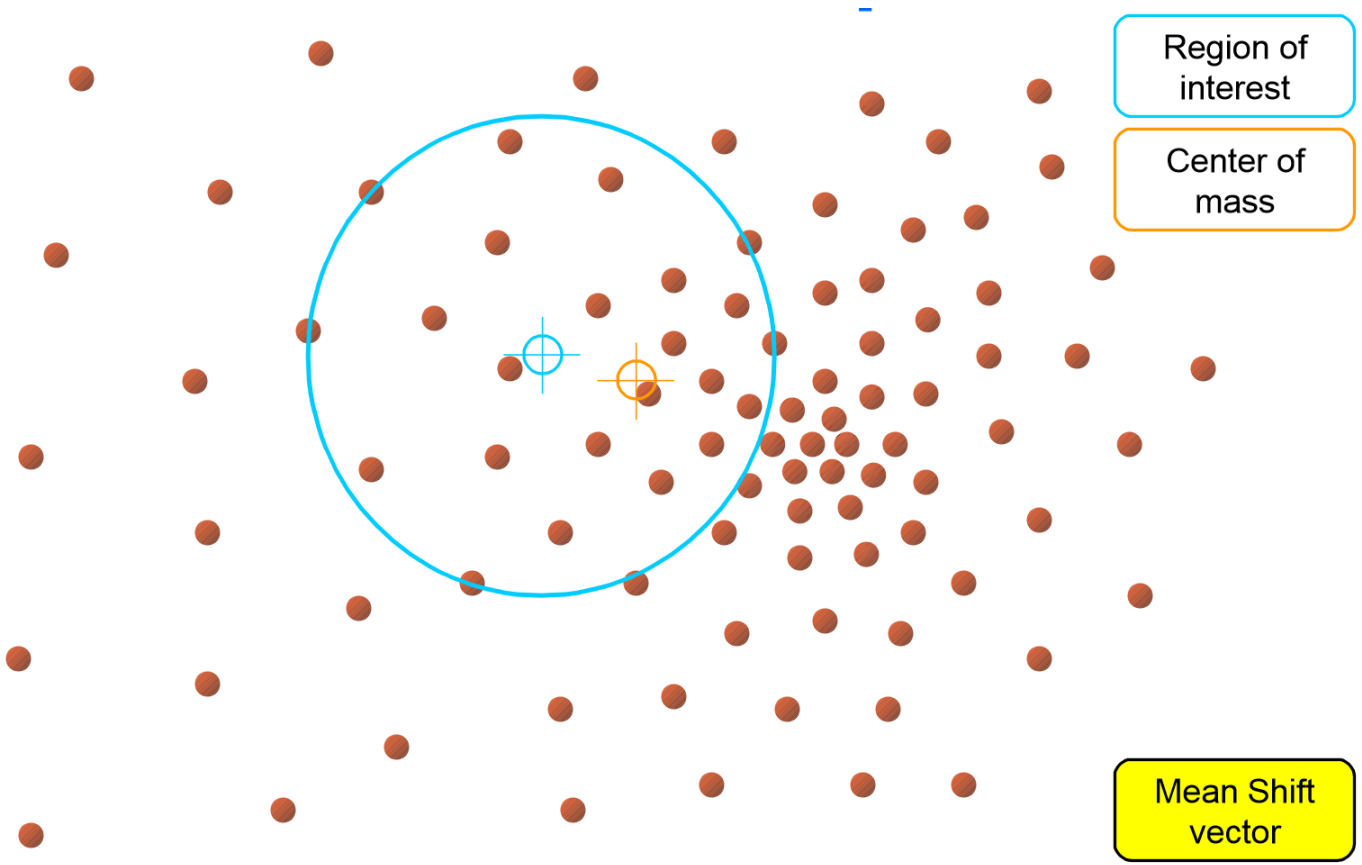


Mean shift clustering



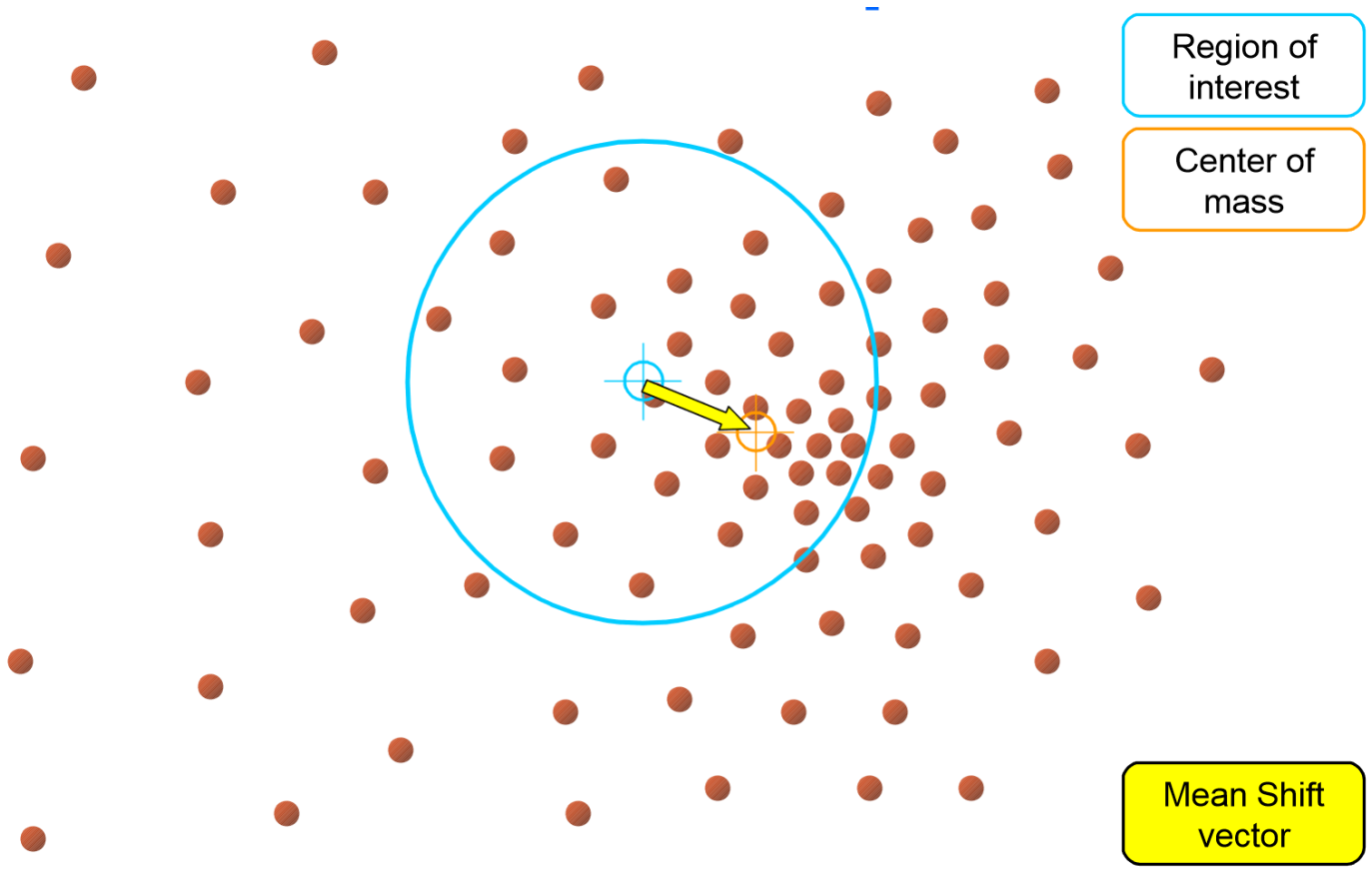
Objective : Find the densest region

Mean shift clustering



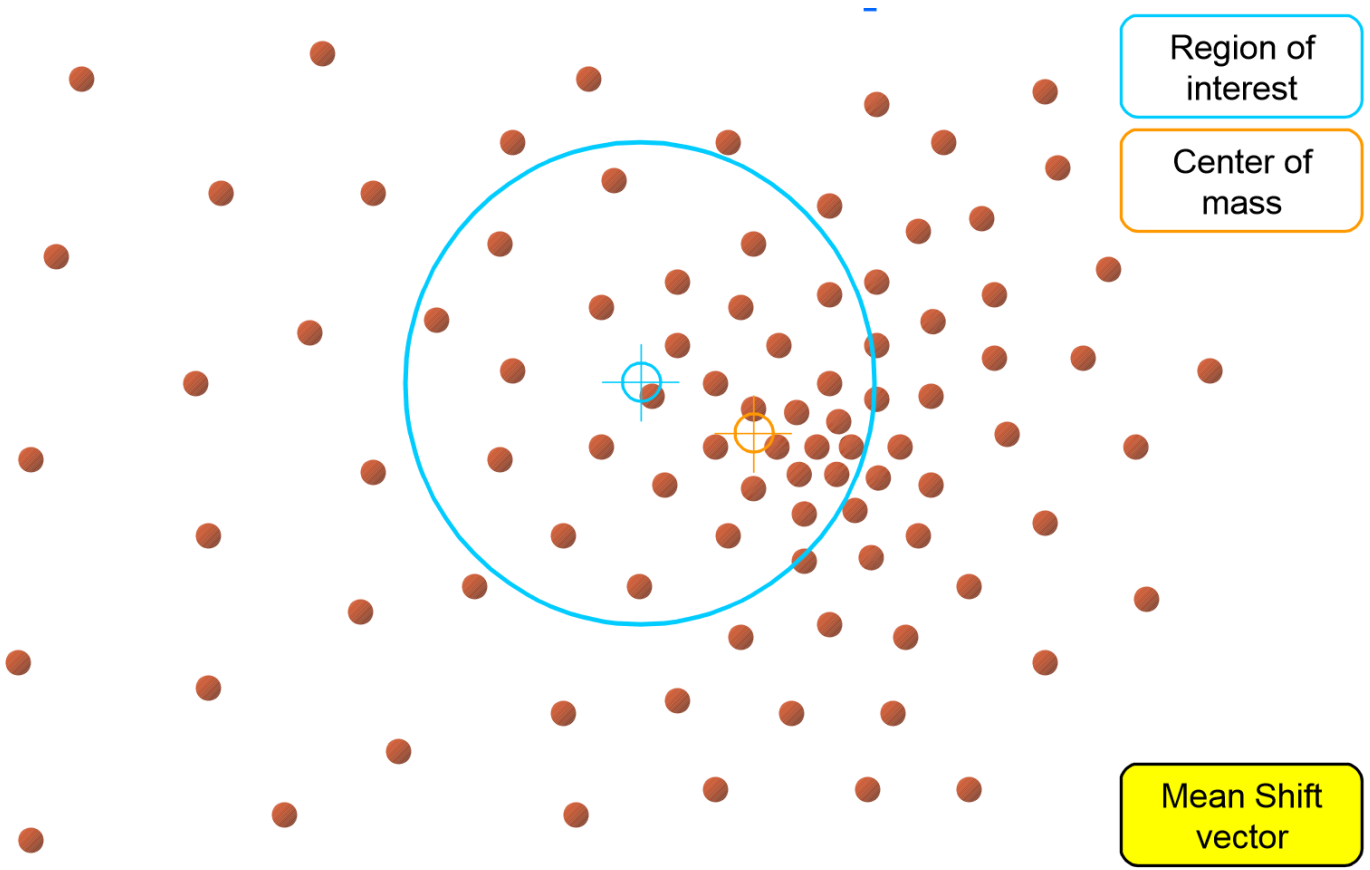
Objective : Find the densest region

Mean shift clustering



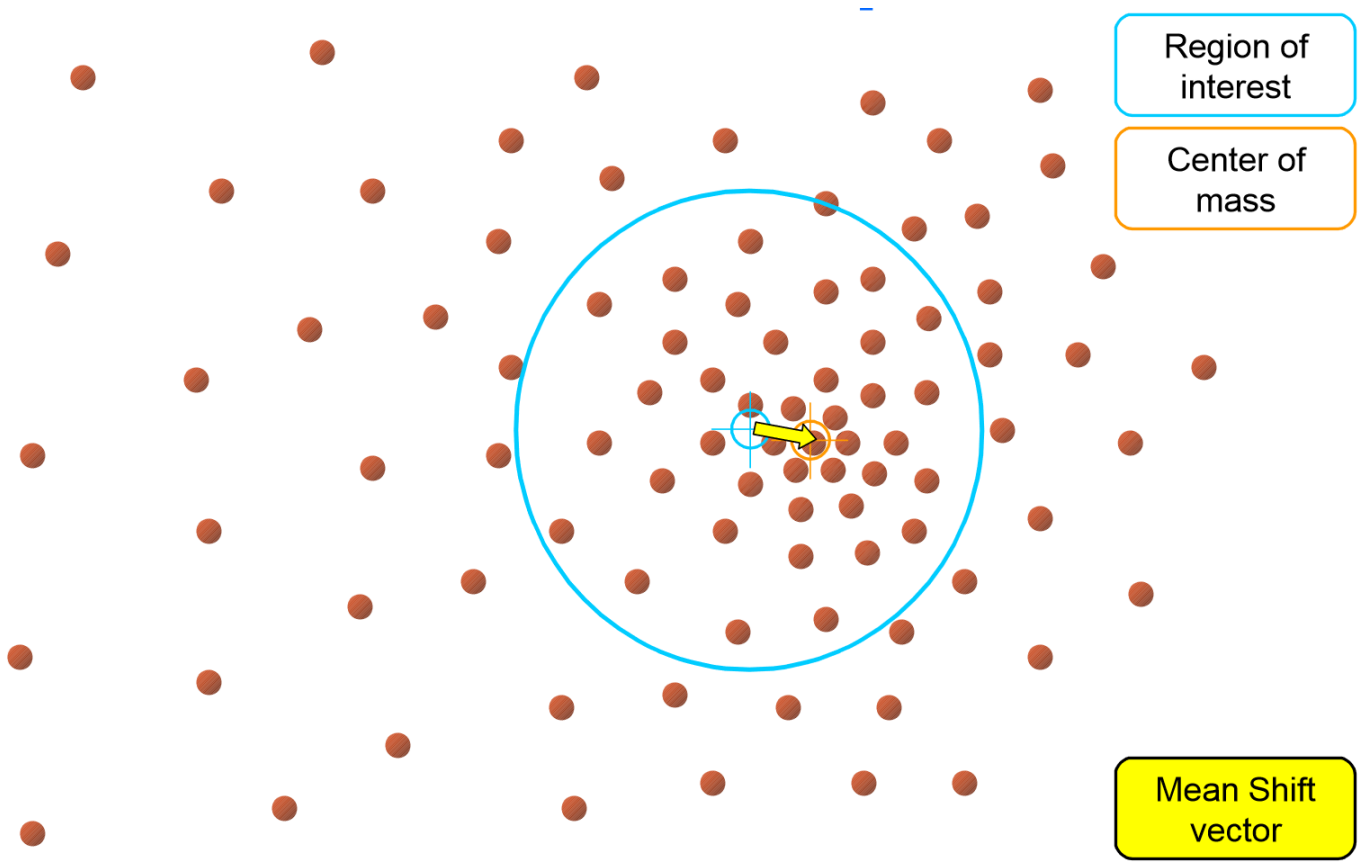
Objective : Find the densest region

Mean shift clustering



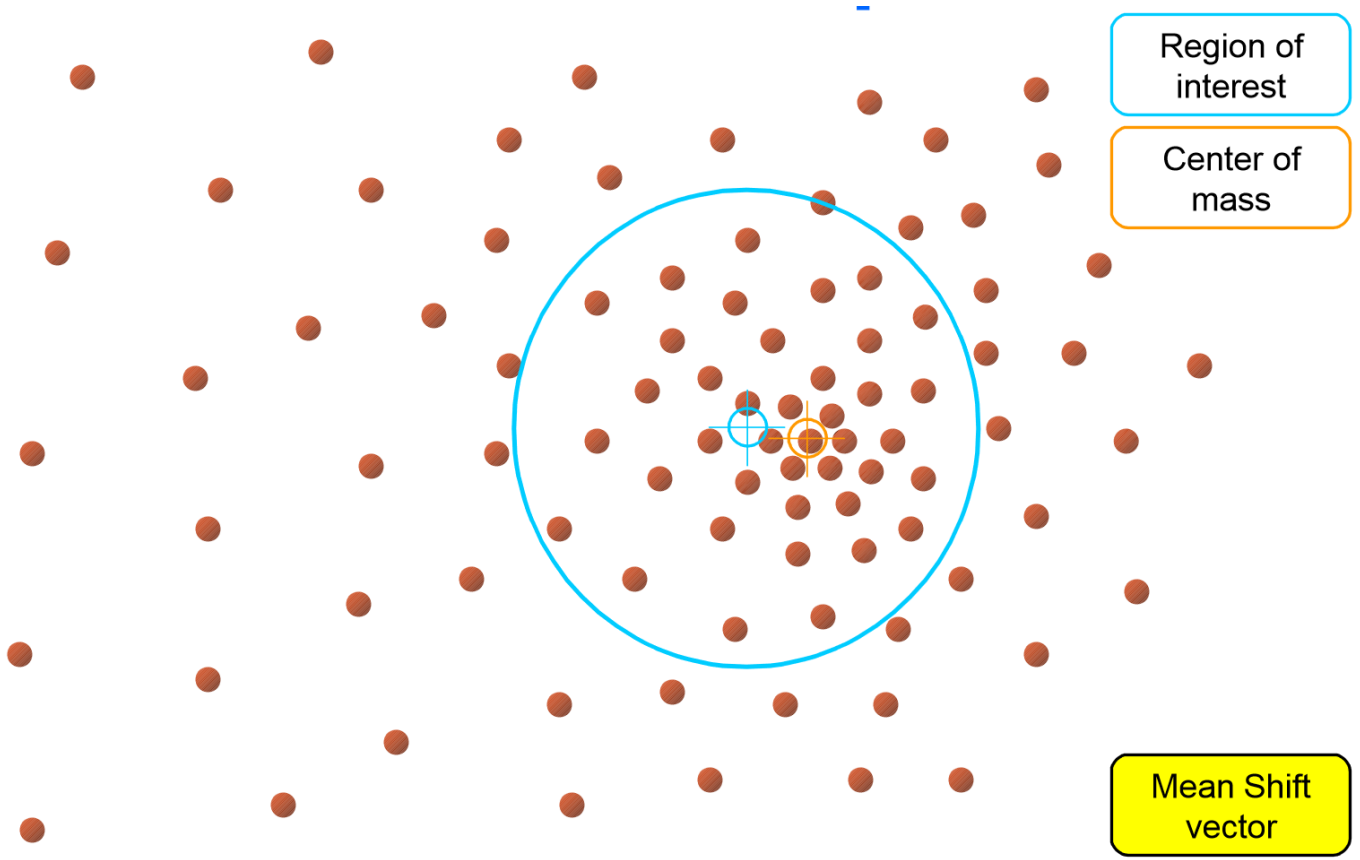
Objective : Find the densest region

Mean shift clustering



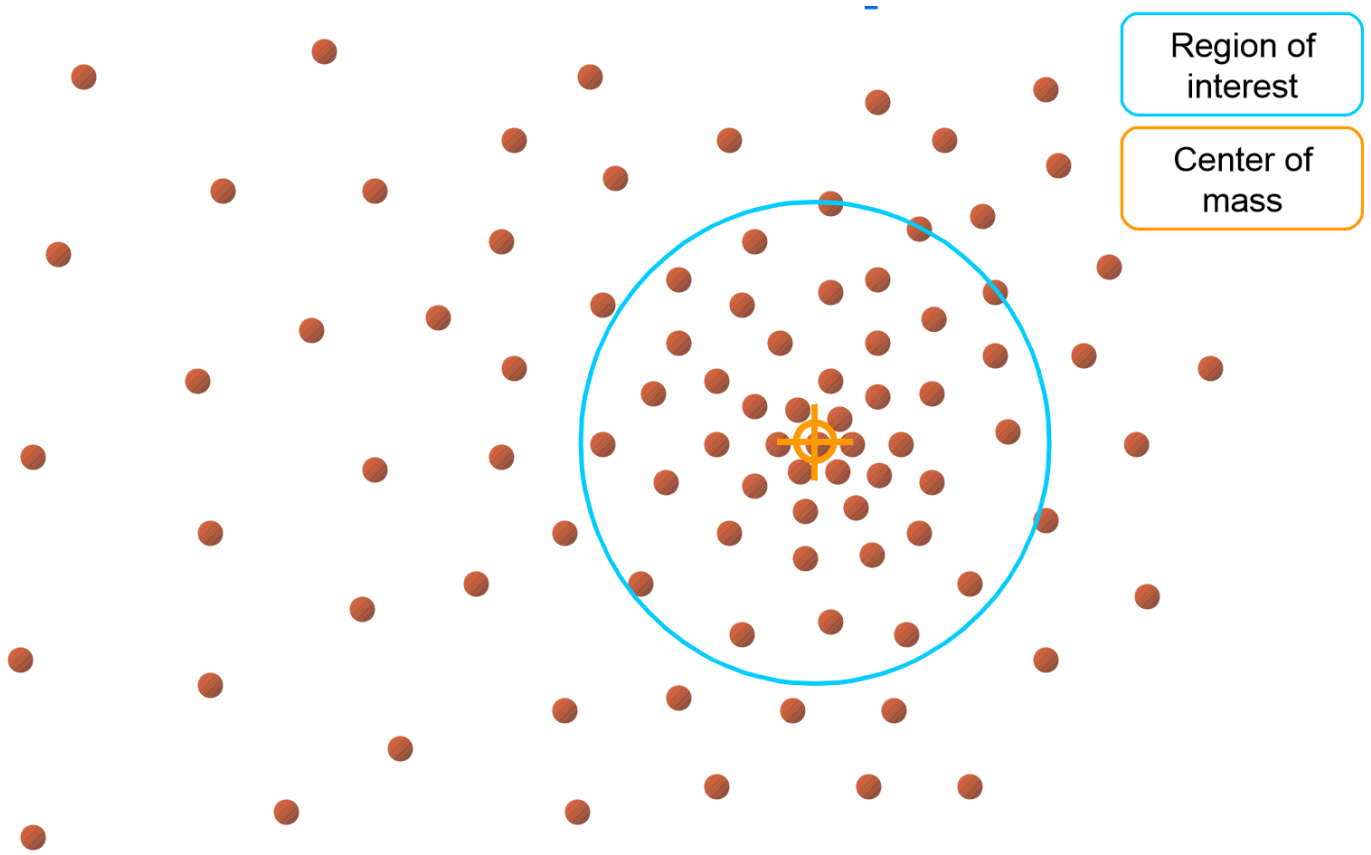
Objective : Find the densest region

Mean shift clustering



Objective : Find the densest region

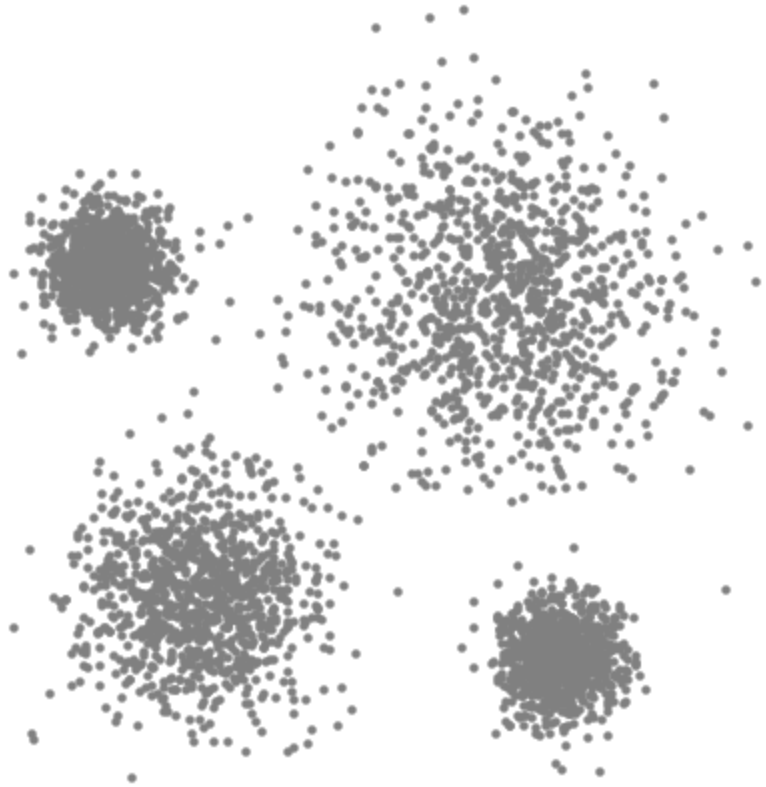
Mean shift clustering



Objective : Find the densest region

AALBORG UNIVERSITY
DENMARK

Mean-shift clustering



AALBORG UNIVERSITY
DENMARK

Illustration: https://towardsdatascience.com/the-5-clustering-algorithms-data-scientists-need-to-know-a36d136ef68

Mean-shift clustering

Pros:

- Does not need a predefined number of clusters

Cons:

- Need to choose the window size/radius
- Detects spherical clusters only

DBSCAN

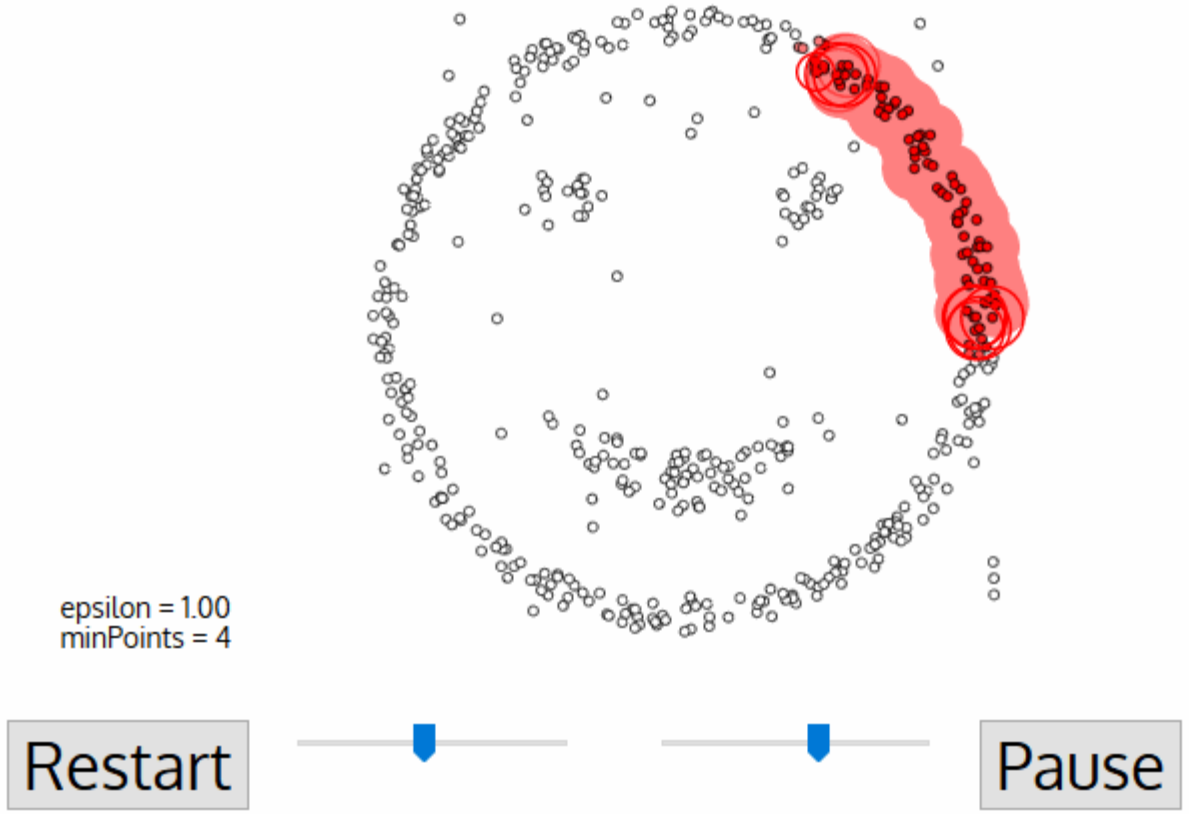
Density-based spatial clustering of applications with noise (DBSCAN)

Algorithm:

1. Compute neighbors within ε distance of each point and identify core points with more than minPts neighbors
2. Join neighboring core points into clusters
3. Foreach non-core point do
   - Add to a neighboring core point if possible
   - Otherwise, add to noise



DBSCAN



AALBORG UNIVERSITY
DENMARK

Illustration: https://towardsdatascience.com/the-5-clustering-algorithms-data-scientists-need-to-know-a36d136ef68

DBSCAN

Pros:

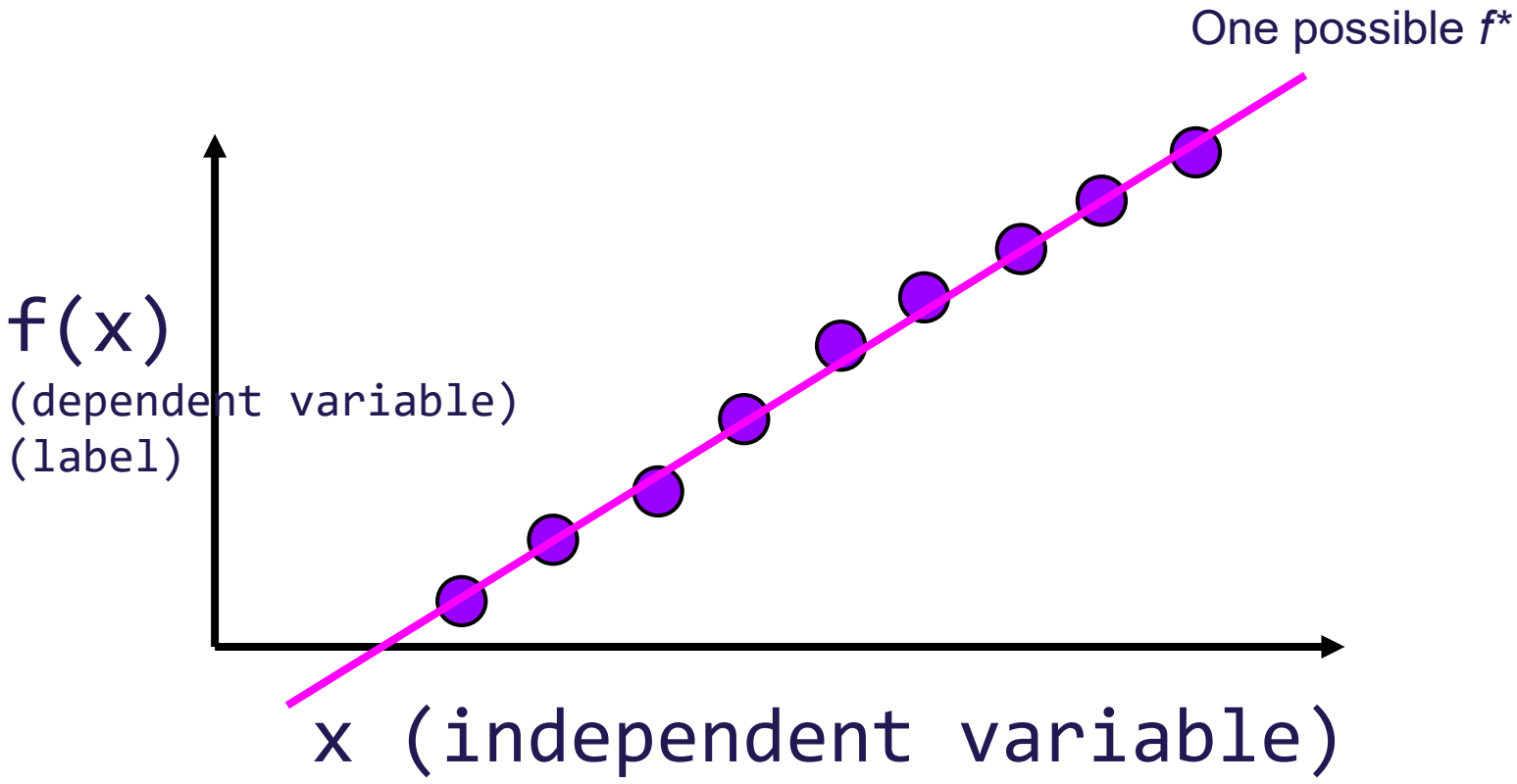
- Does not need a predefined number of clusters
- Can find arbitrarily shaped and sized clusters
- Can handle noise/outliers

Cons:

- Cannot cluster data sets well with large differences in densities
  - Need to choose ε and minPts

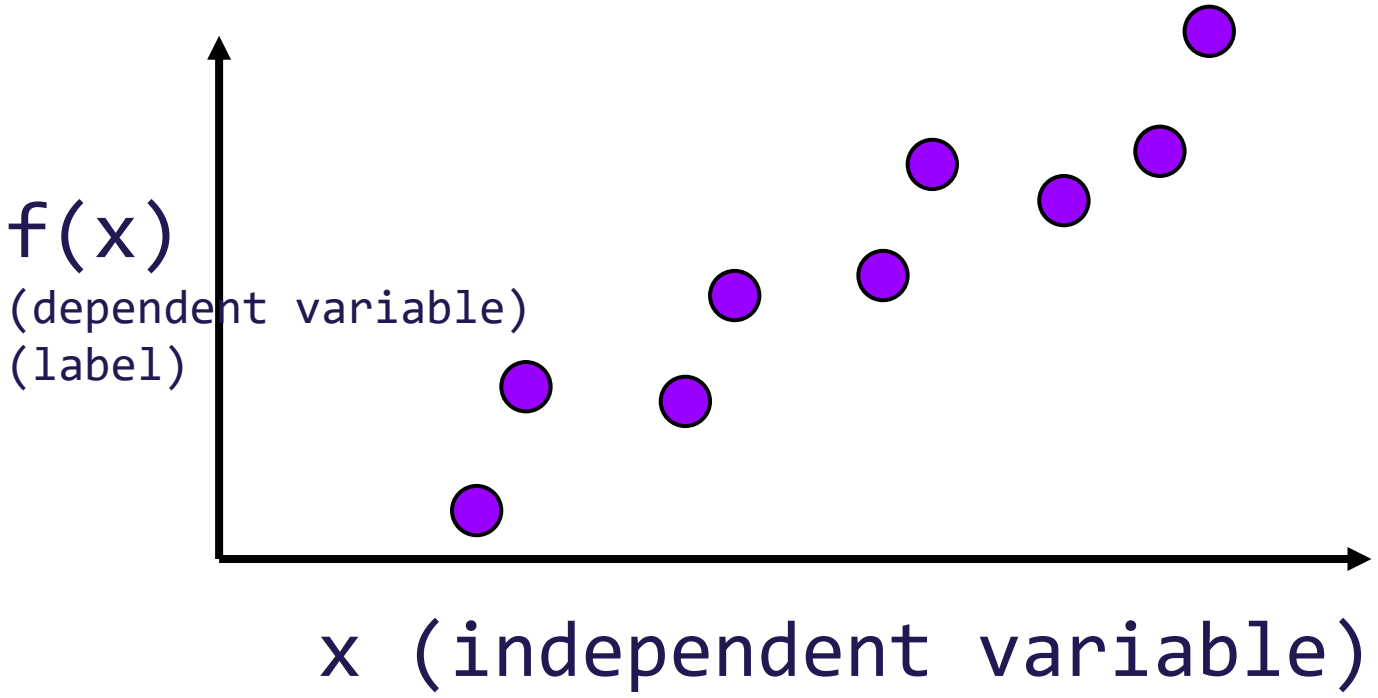
Supervised learning

- Want to estimate underlying function *f*



Supervised learning

Data often has noise



Supervised learning

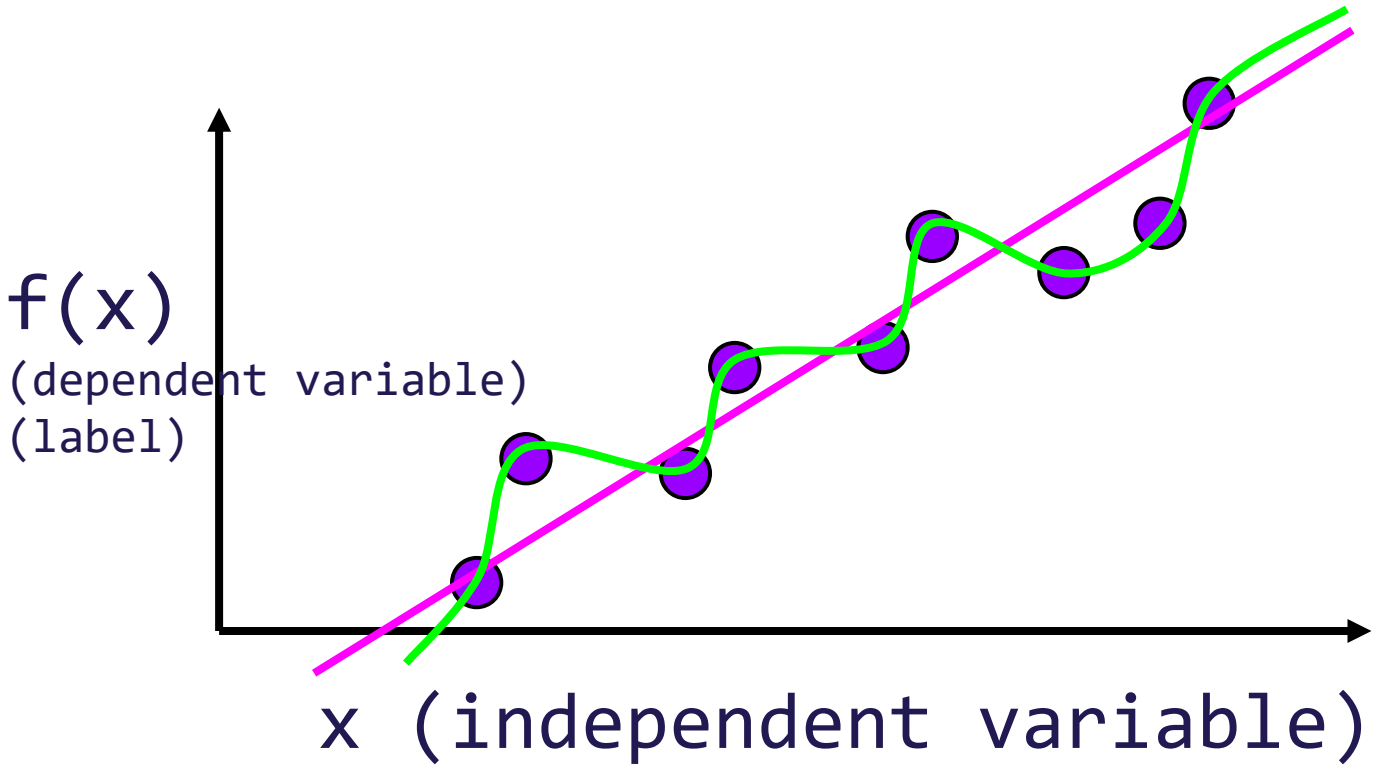
Data often has noise

Why?

- Randomness
  - Random distribution of photons hitting sensor, sensors aren’t precise
- Mislabelled data
  - Common when humans label lots of data
- Variables outside of model
  - Variations might look like noise but are explained by a hidden, unknown variable

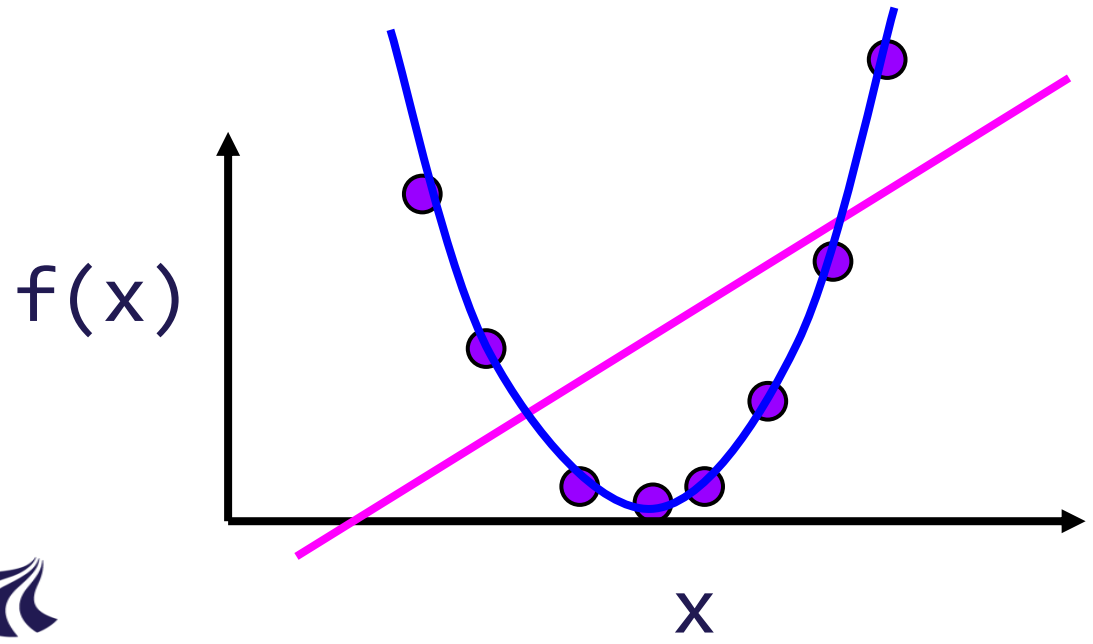
Supervised learning

Which model is right?



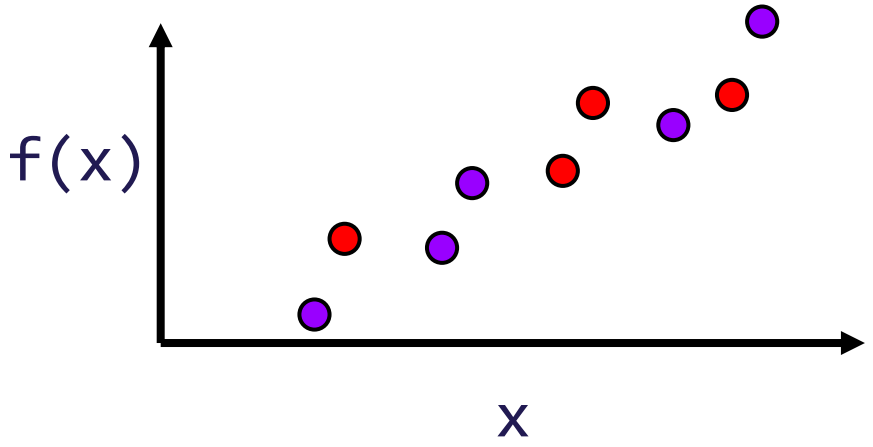
Supervised learning

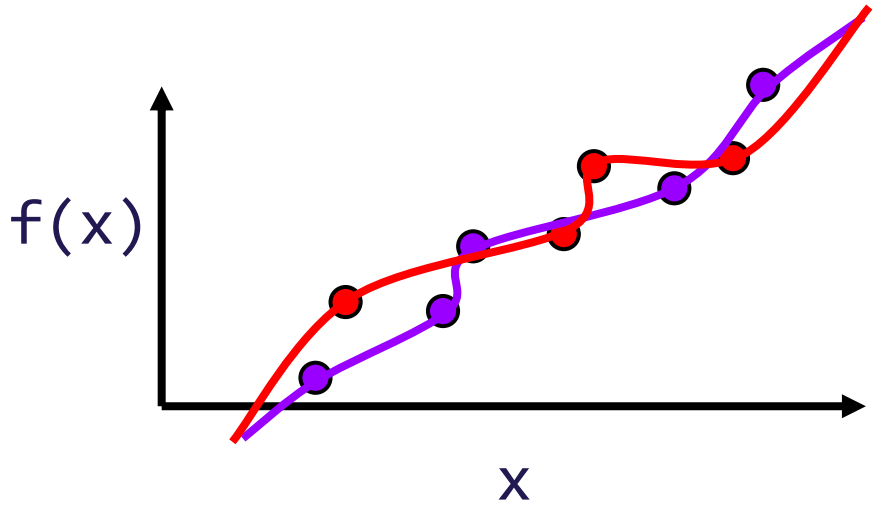
- Bias
  - Error from assumptions model makes about data
  - Linear model assumes data is linear, bad for data that isn't
  - Quadratic is better



Supervised learning

- Variance
  - Algorithm's sensitivity to noise
  - More complex algorithms are more sensitive!
  - High variance hurts generalization, *overfitting!*





Supervised learning

Noise
- Random variations in data

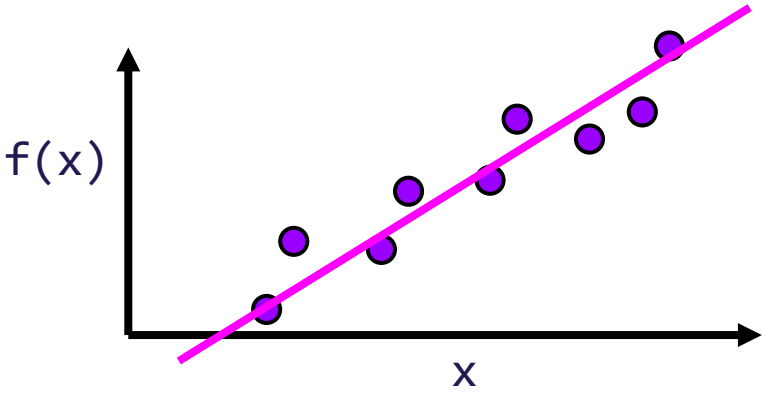
Bias
- Error from assumptions model makes about data
- Less complex algorithms -> more assumptions about data

Variance
- Algorithm’s sensitivity to noise
- More complex algorithms are more sensitive!
- High variance hurts generalization

Tradeoff between bias and variance!

Linear regression

- f*(x) = ax + b
- Learn a and b from data (how?)
  - Minimize squared error!
  - Loss function $L(f^*) = \sum_i \|f(x_i) - f^*(x_i)\|^2$
  - Want $\underset{a,b}{\operatorname{argmin}}[L(f^*)]$
  - Extrema when derivative = 0
  - Solve linear system of equations
    - $\boldsymbol{Ma = b}$



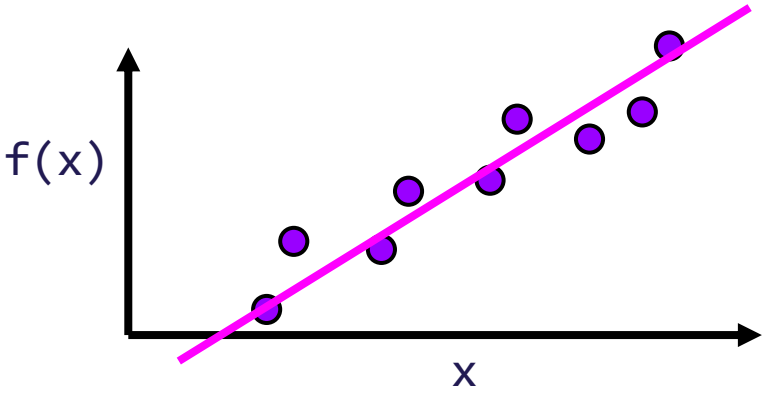
Linear regression

- f*(x) = ax + b
- Learn a and b from data

- High bias: linear assumption
- Low variance

- Benefits:
  - Closed form solution
  - Fast to compute for new data

- Weaknesses:
  - Not very powerful, assumes linear relation
  - Underfit more interesting data



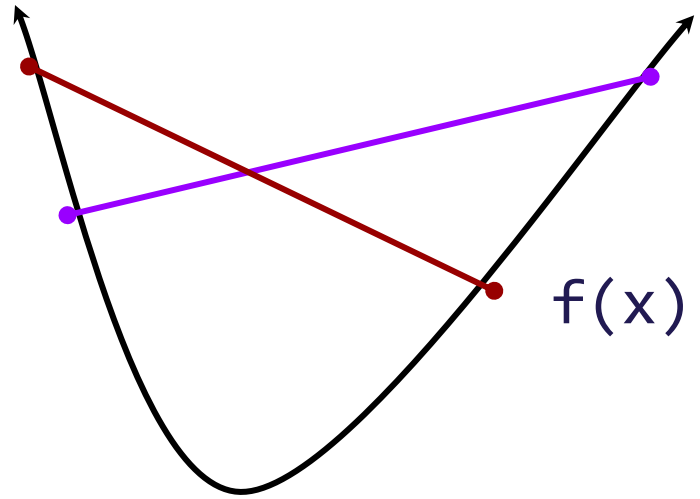
Loss function

Convex vs. non-convex

- Convex function: connect any two points on graph with a line, that line lies above function everywhere

Why is it important? Any local extrema is global extrema!

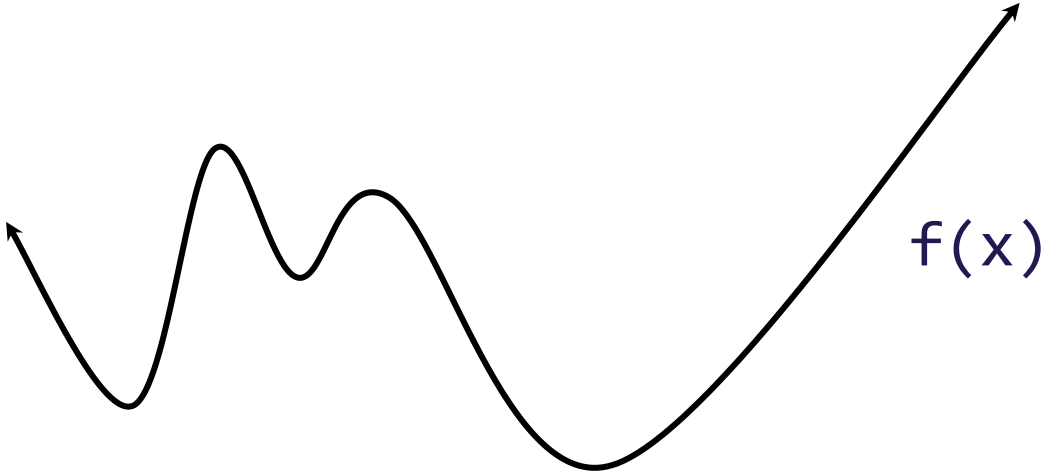
- If our loss function is convex, can set derivative = 0, solve for parameters (sometimes still no closed-form)



Loss function

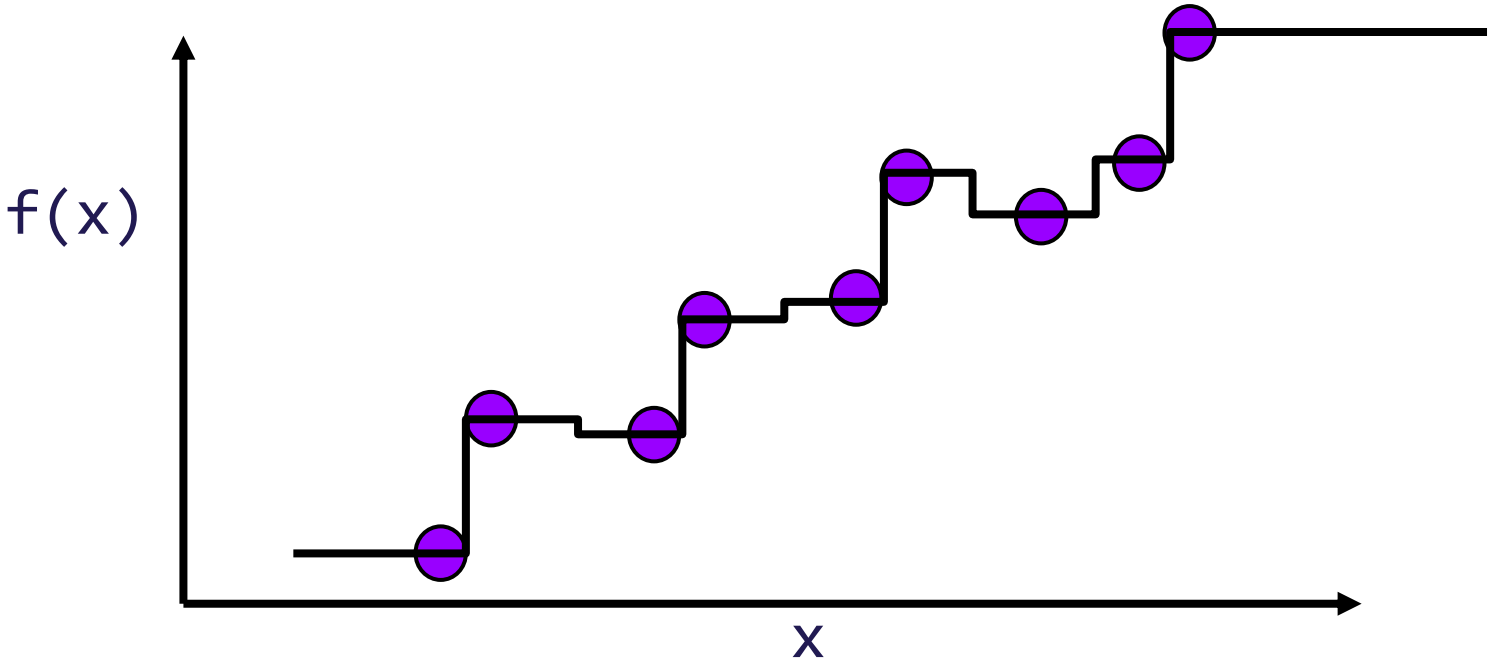
Convex vs. non-convex

- Non-convex function: no rules!
- Local optima are not global optima
- Usually no easy way to find global or local optima, harder to optimize



Nearest neighbor

- f*(x) = f(x') for nearest x' in training set



Nearest neighbor

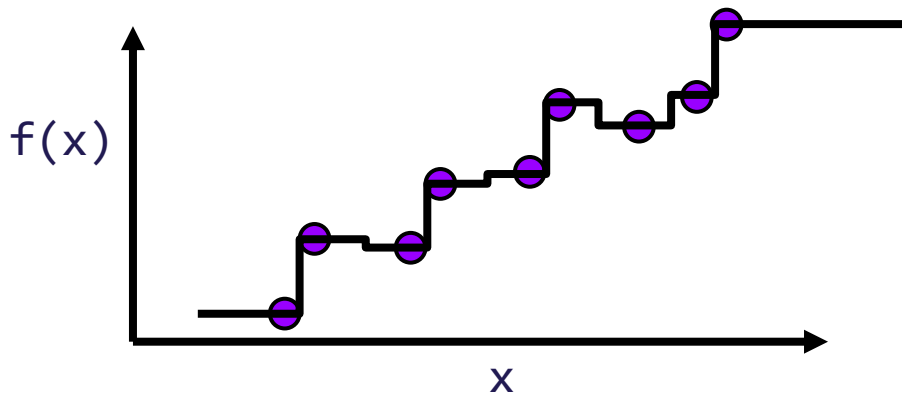
- f*(x) = f(x') for nearest x' in training set
- Low bias: no assumptions about data
- High variance: very sensitive to training set

Benefits:

- Super easy to implement
- Easy to understand
- Arbitrarily powerful, esp. with lots of data

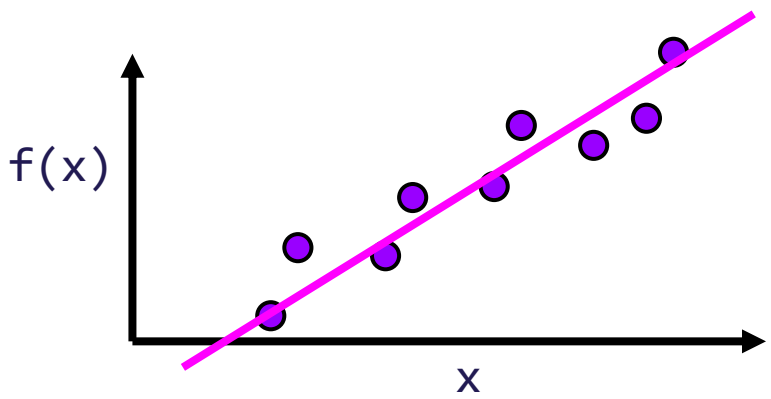
Weaknesses:

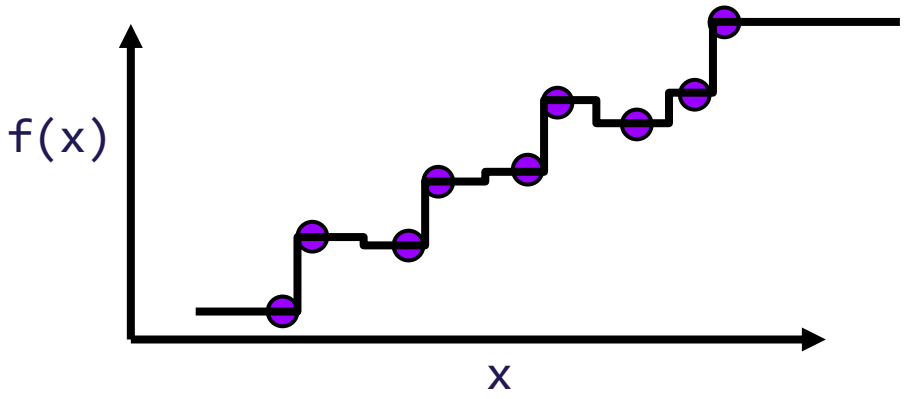
- Hard to scale
- Prone to overfitting to noise



Regression

- Given training data
  - input variables X, output variables Y
- And new data point x’
- Predict corresponding output variables y’





Classification

- Training data: points associated with a class
  - Also other data about that point
- Toy example: Does patient have the flu?
  - Binary classification (yes or no)
  - Different types of variables (continuous, discrete)

sore throat	runny nose	nausea	temp	chills	pain	age	days	diagnosis
no	yes	yes	101.3	yes	7	15	5	flu
yes	yes	no	98.8	no	3	74	3	not flu
yes	yes	no	100.1	yes	4	46	4	flu
yes	yes	yes	99.8	yes	6	27	1	flu
yes	no	no	98.4	yes	5	35	2	not flu
yes	yes	yes	99.0	no	3	42	4	not flu

Classification

One approach: partitions

- Find best split or splits to data along one variable
- One possibility, Pr(flu) = (temp > 99.5)
  - Pretty accurate on our training data

sore throat	runny nose	nausea	temp	chills	pain	age	days	diagnosis
no	yes	yes	101.3	yes	7	15	5	flu
yes	yes	no	98.8	no	3	74	3	not flu
yes	yes	no	100.1	yes	4	46	4	flu
yes	yes	yes	99.8	yes	6	27	1	flu
yes	no	no	98.4	yes	5	35	2	not flu
yes	yes	yes	99.0	no	3	42	4	not flu

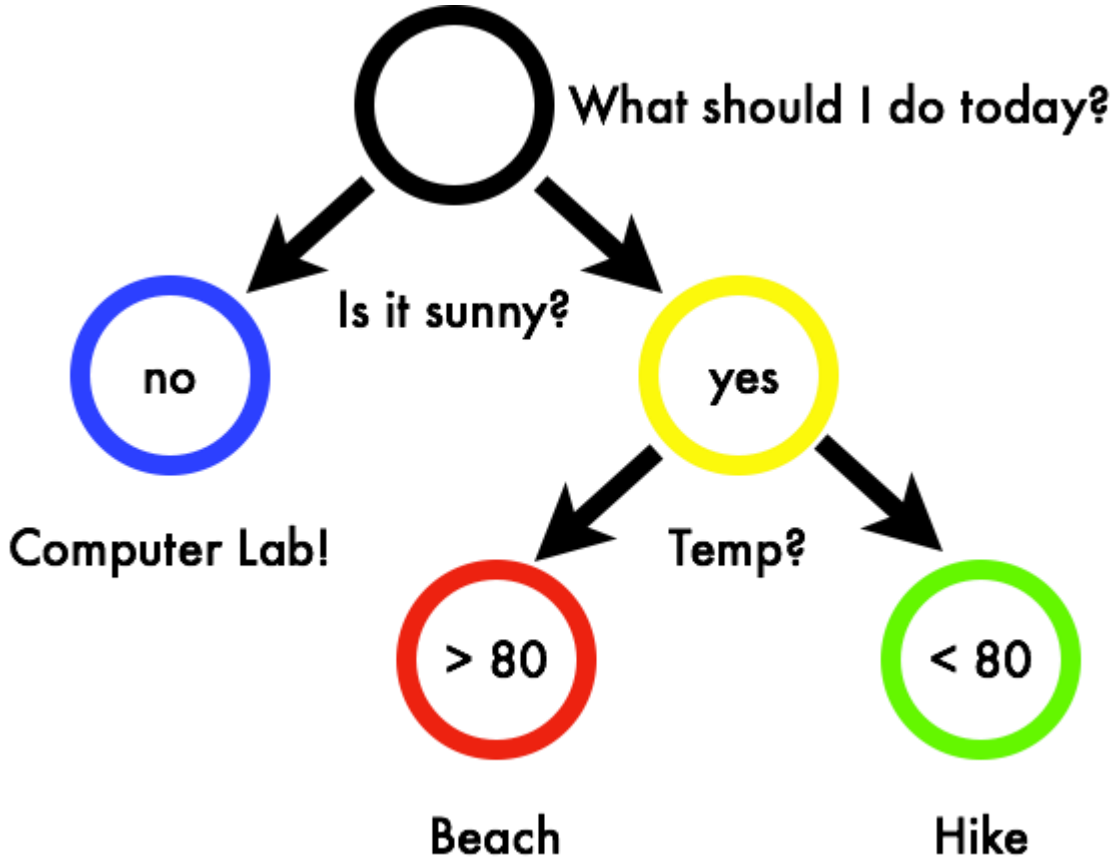
Tree classifiers

Trees: layers of partitions

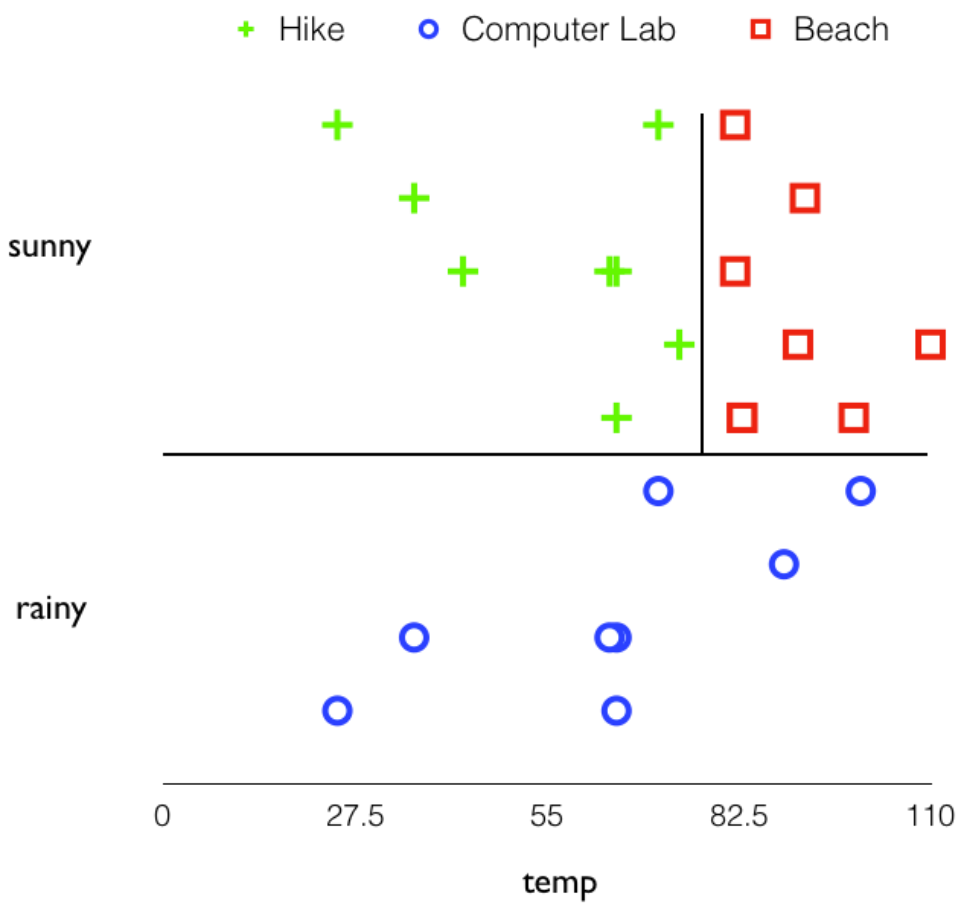
Very simple models

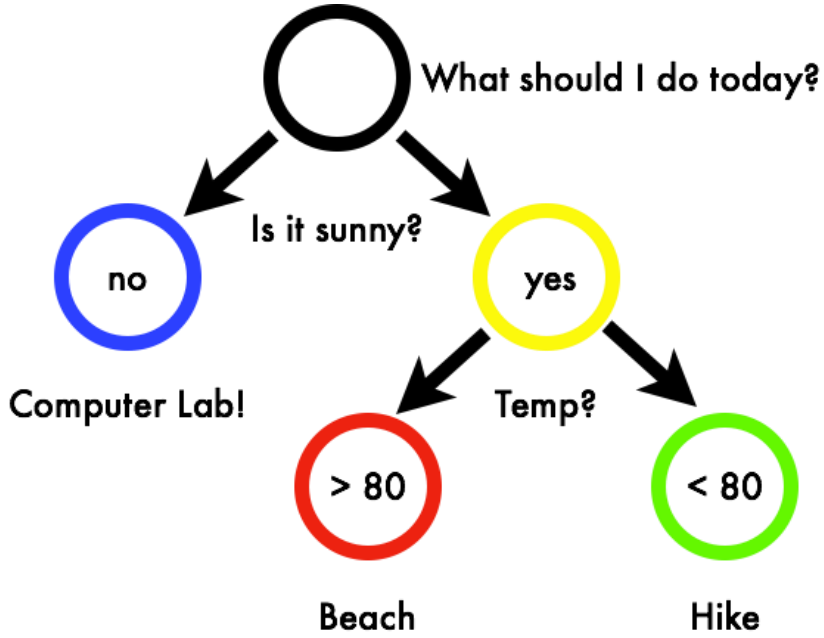
Benefits:

- Interpretable
- Easy to use
- Good for applications

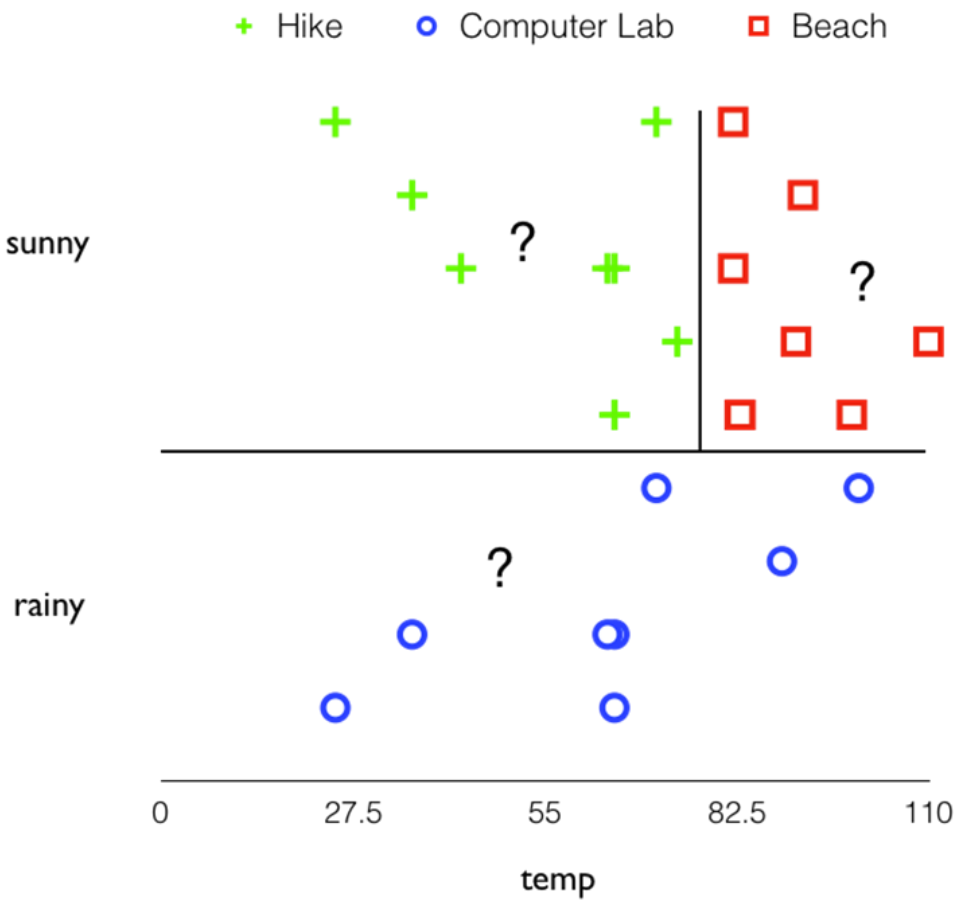


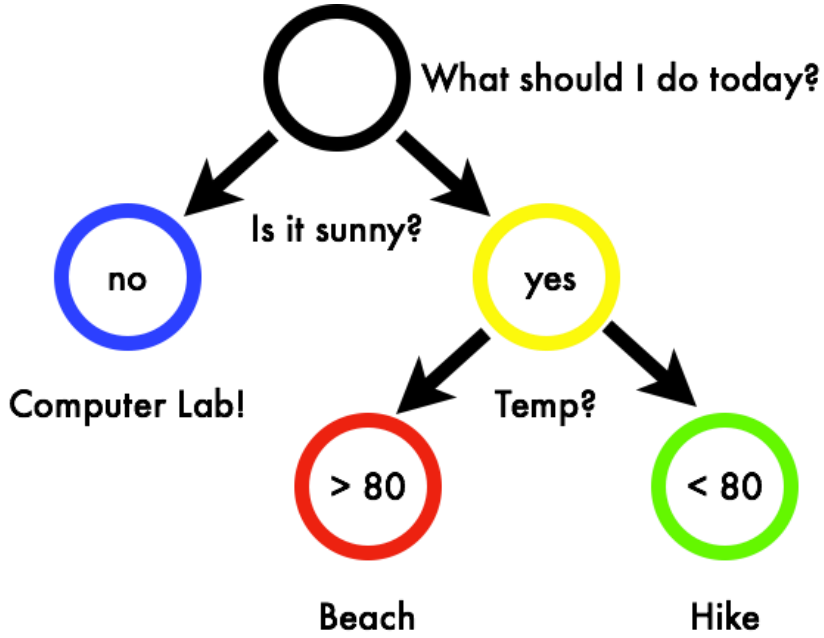
Tree classifiers



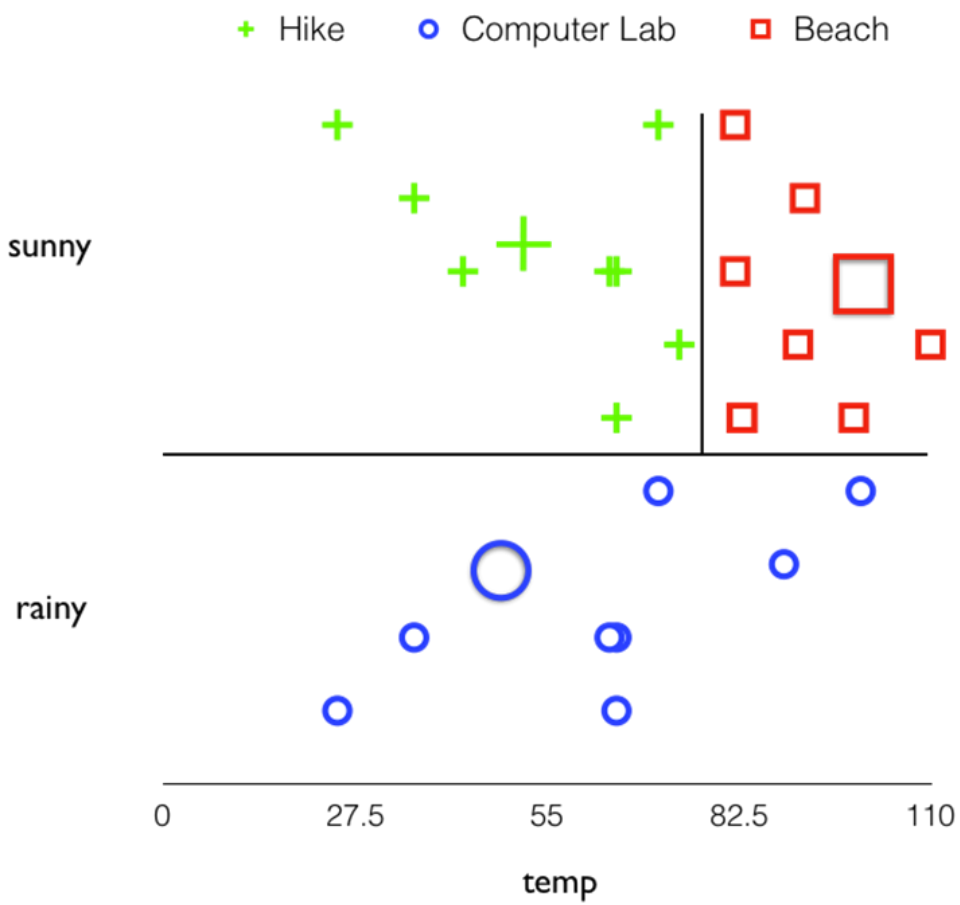


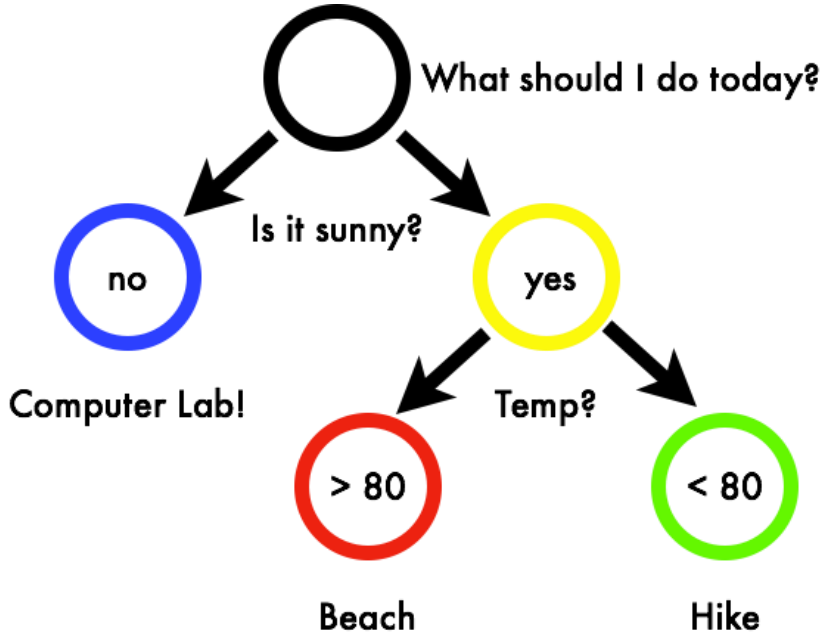
Tree classifiers



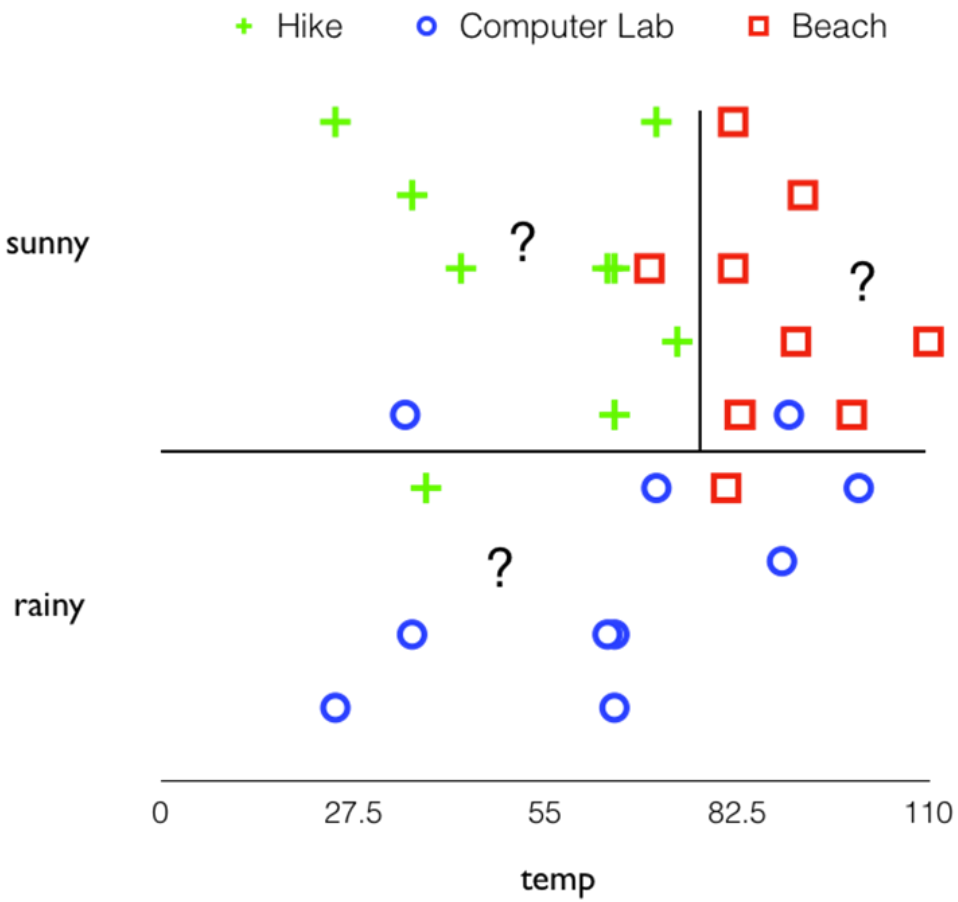


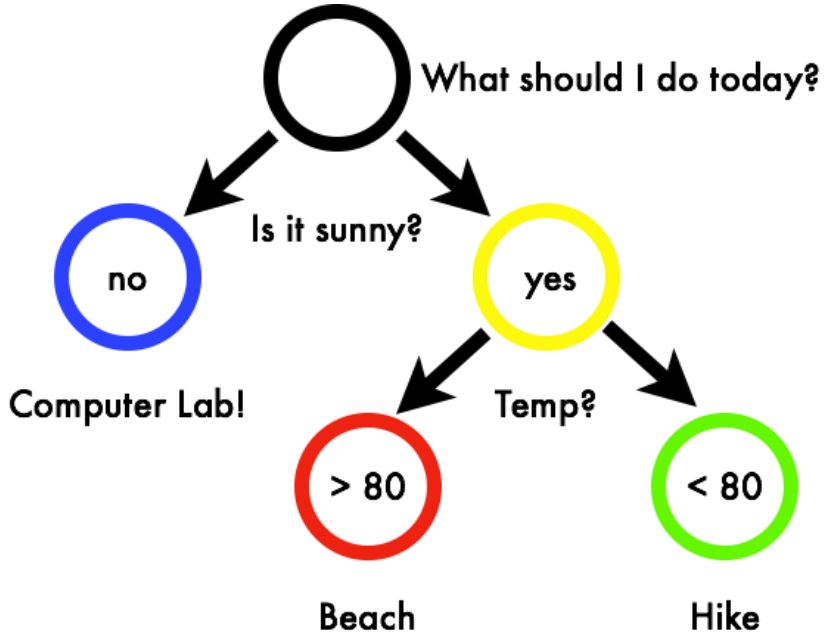
Tree classifiers



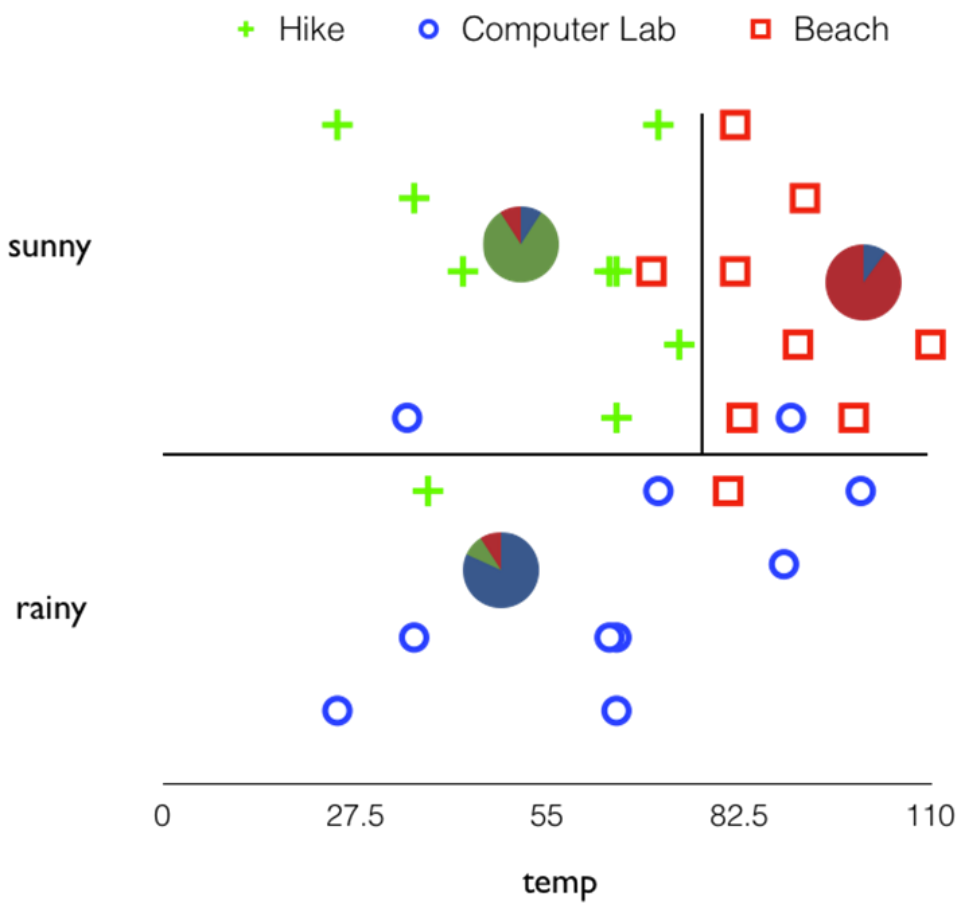


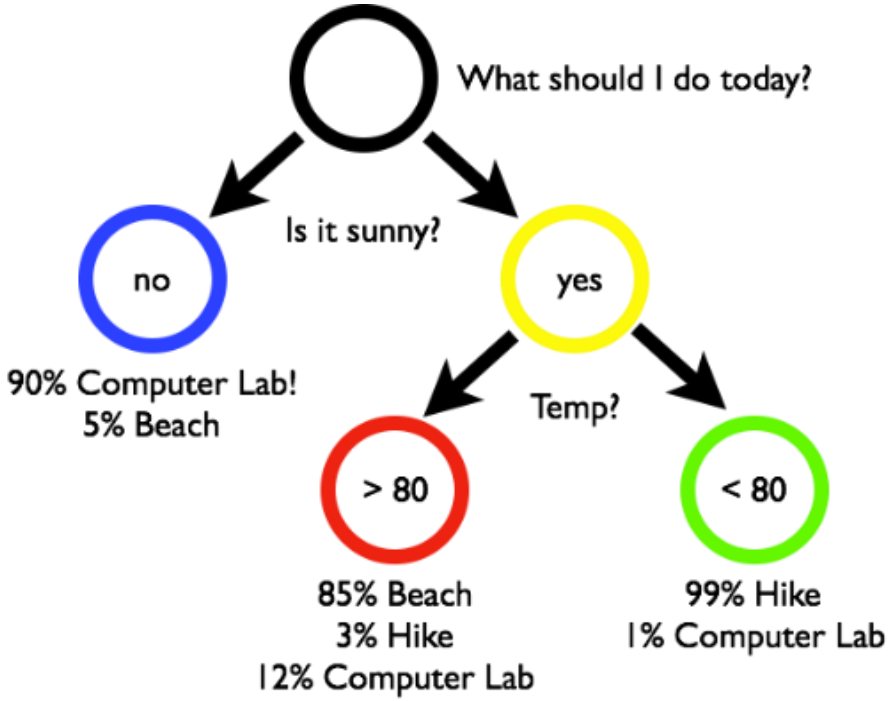
Tree classifiers





Tree classifiers





Random forest



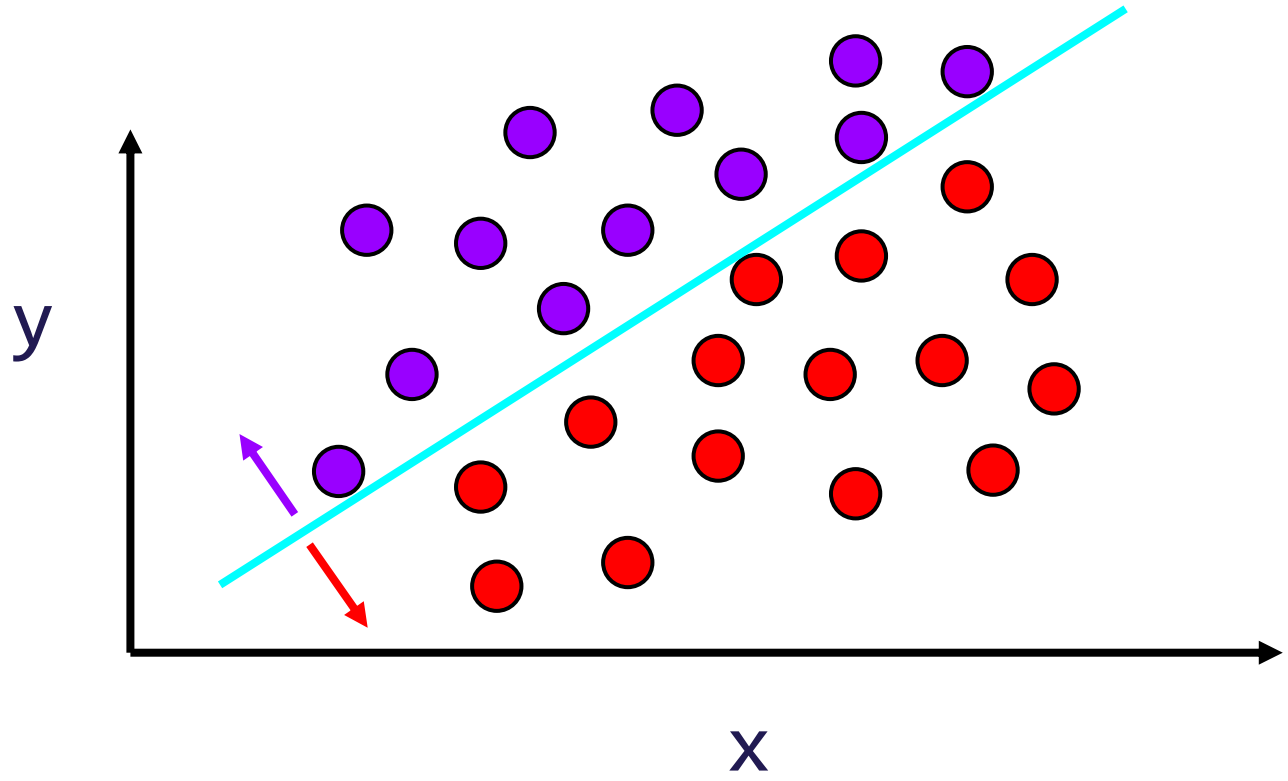
Ensemble classifier based on several decision trees

Algorithm overview:

1. If the number of cases in the training set is N, sample N cases at random - but *with replacement*, from the original data. This sample will be the training set for growing the tree.
2. If there are M input variables/features, a number m<<M is specified such that at each node, m variables are selected at random out of the M and the best split on these m is used to split the node. The value of m is held constant during the forest growing.
3. Each tree is grown to the largest extent possible. There is no pruning.
4. Predict new data by aggregating the predictions of the n trees (majority voting for classification, average for regression)

Classification in two dimensions

Linear classifiers

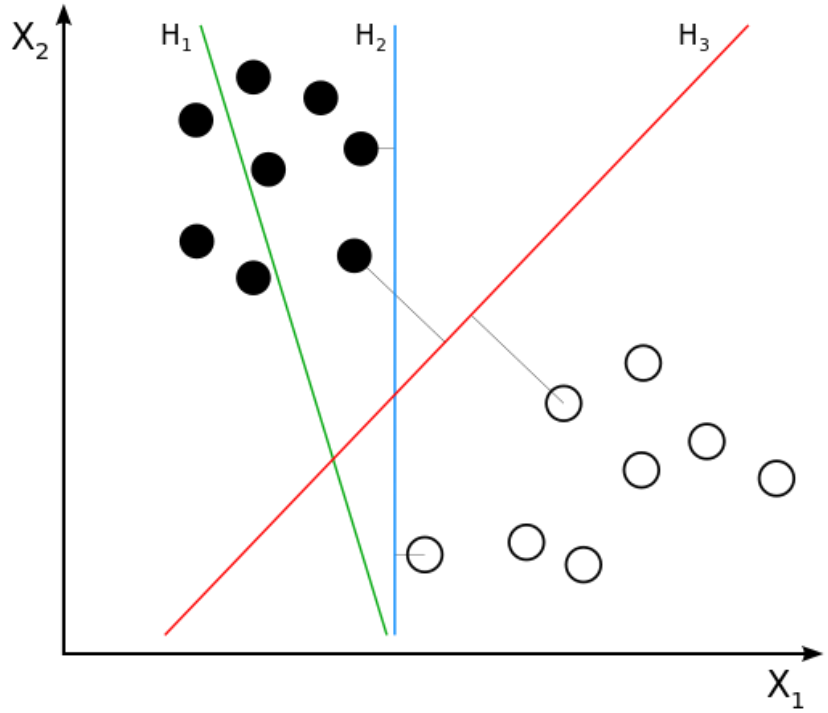


Support Vector Machines (SVM)

Workhorse of old-school vision algorithms! (Example: Human detection with HOG+SVM)

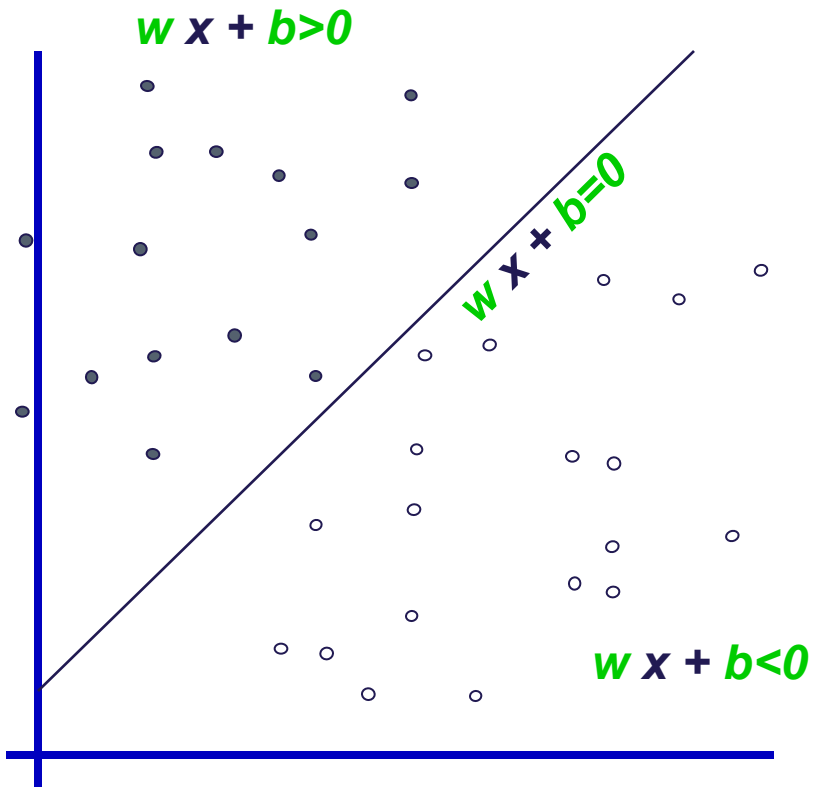
For some data, multiple linear classifiers can separate classes

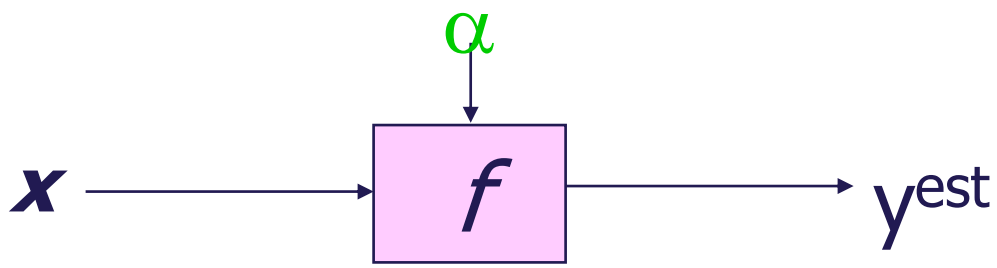
Maybe we want “best” one, maximum margin



Support Vector Machines (SVM)

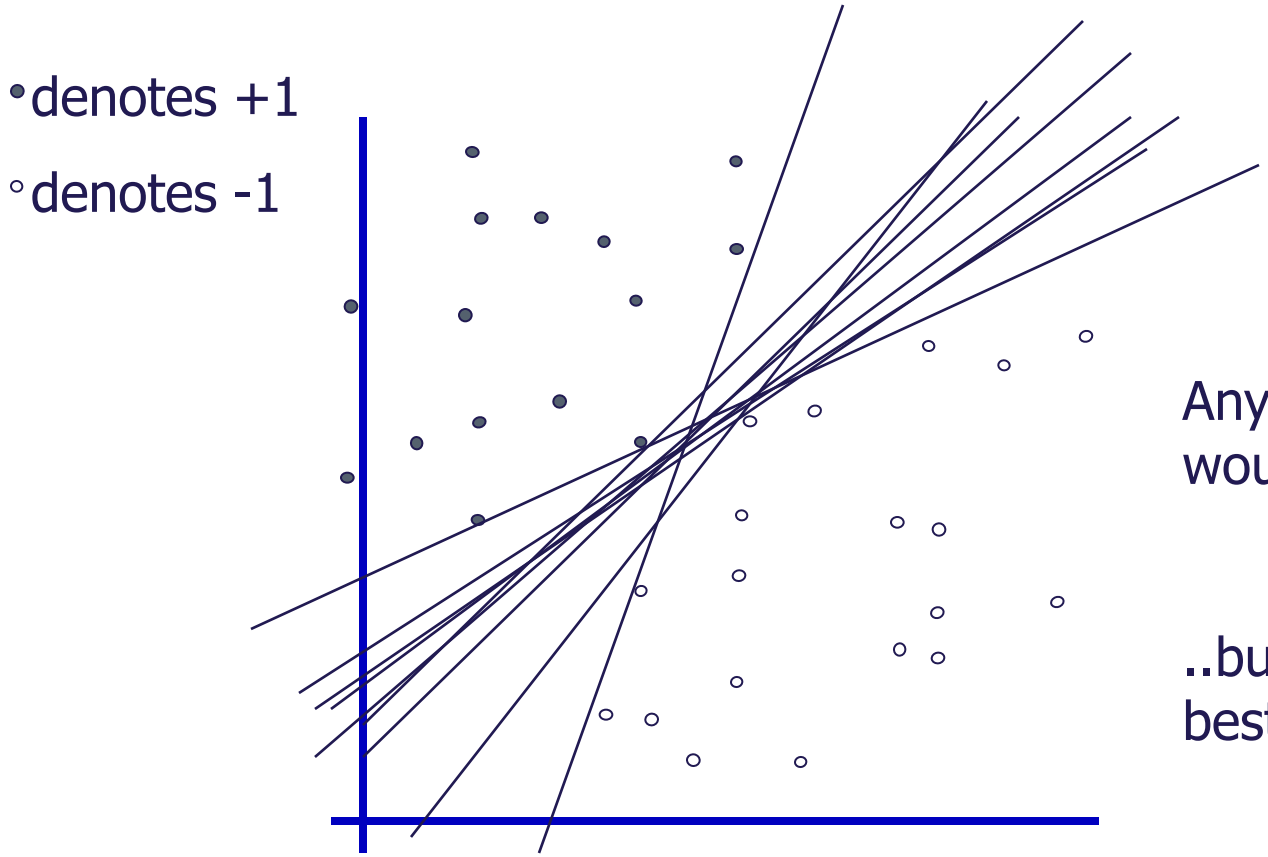
- denotes +1
- denotes -1

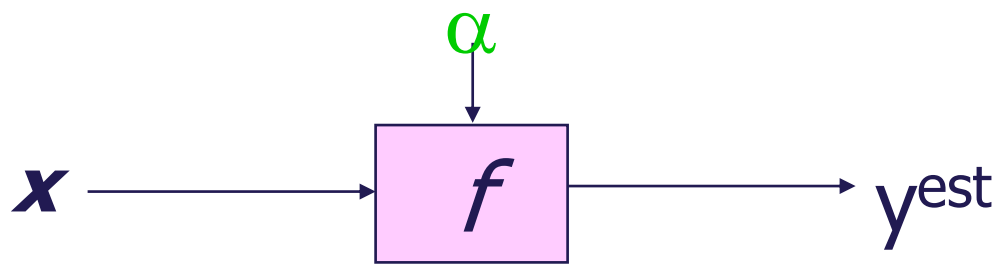




$$f(\mathbf{x},\mathbf{w},b) = sign(\mathbf{w}\,\mathbf{x} + b)$$

Support Vector Machines (SVM)



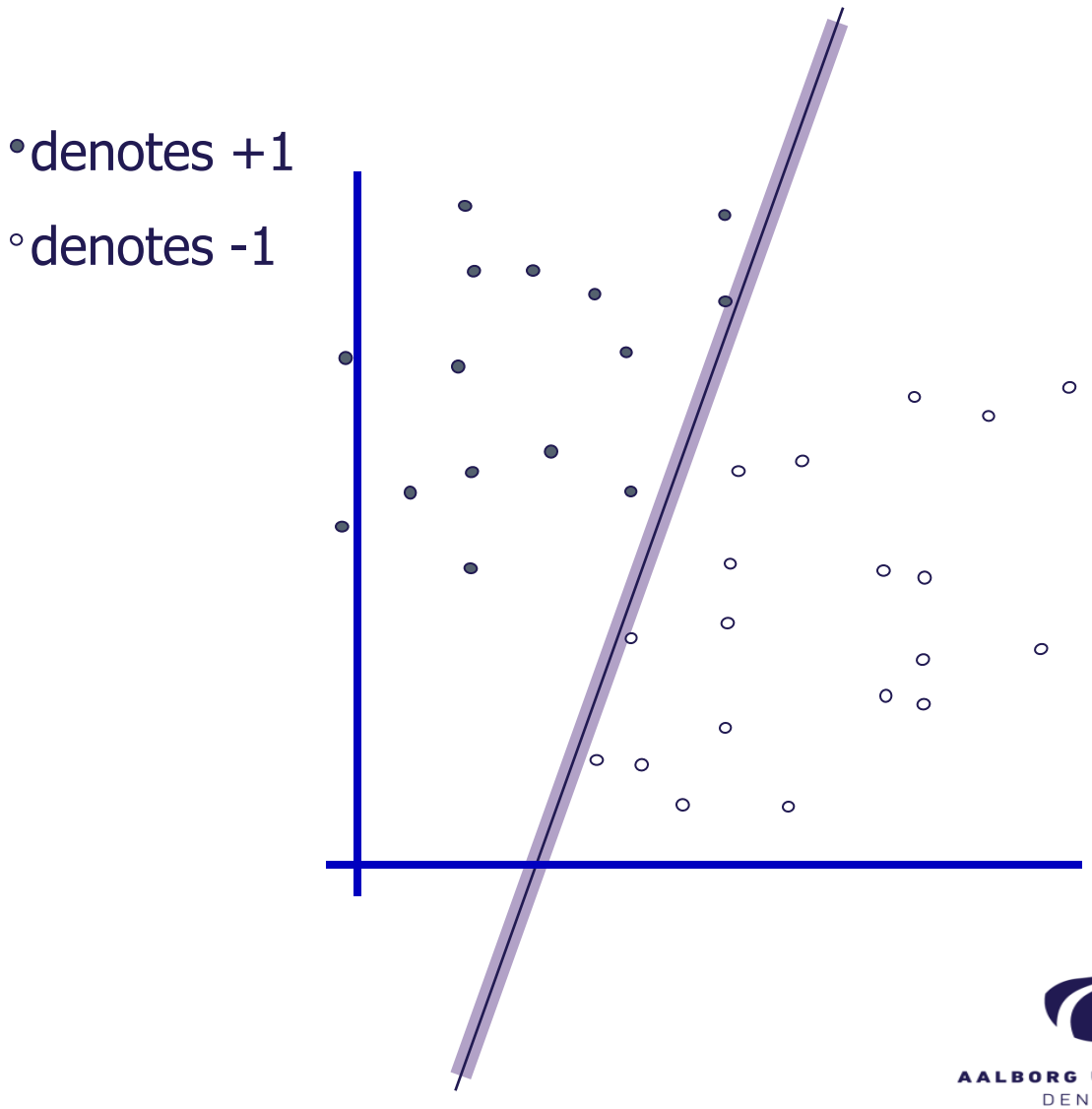


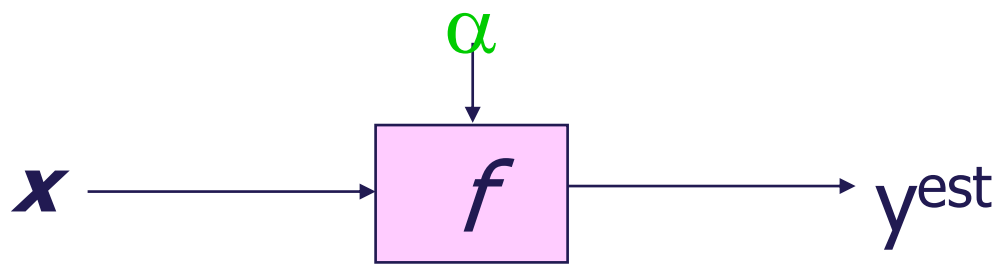
$f(\mathbf{x},\mathbf{w},b) = sign(\mathbf{w}\,\mathbf{x} + b)$

Any of these would be fine..

..but which is best?

Support Vector Machines (SVM)

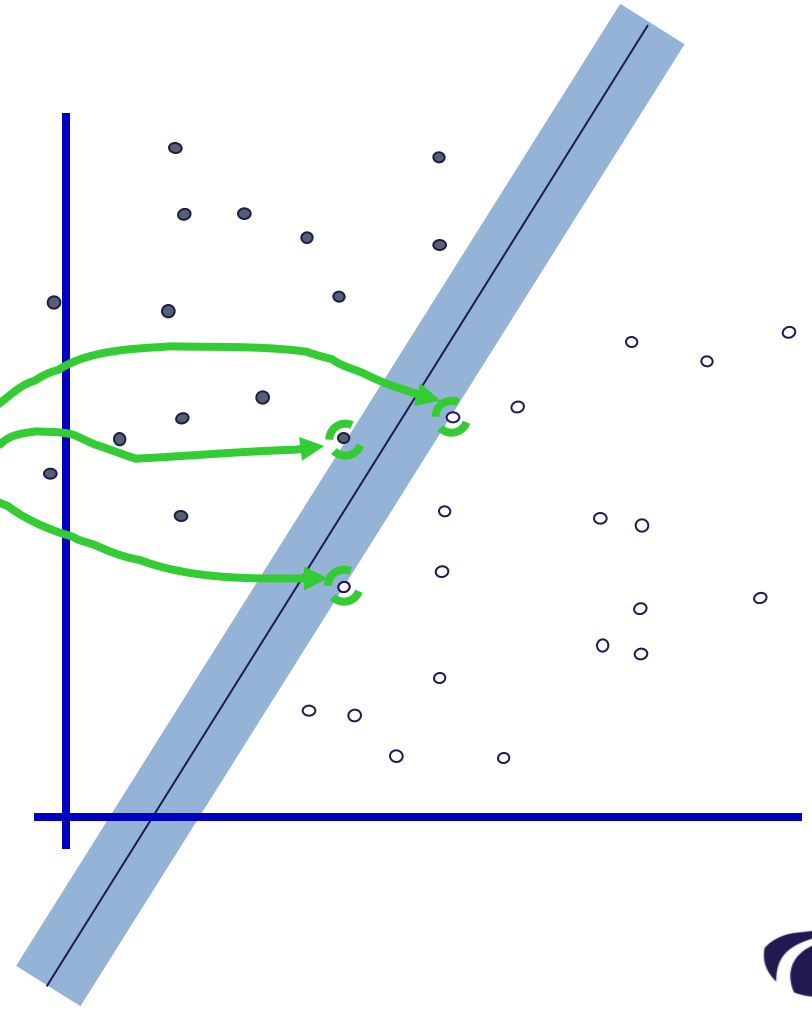


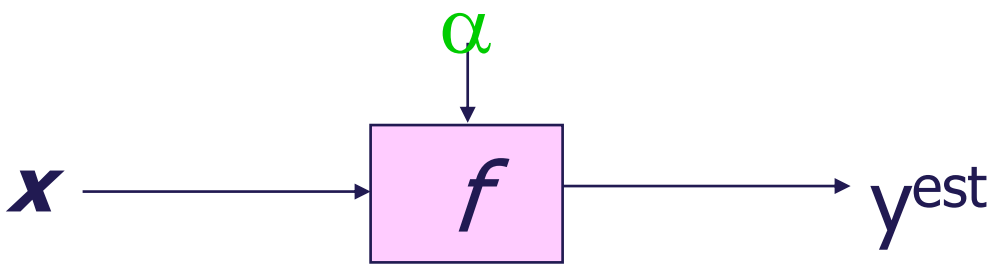


$$f(\mathbf{x},\mathbf{w},b) = sign(\mathbf{w}\,\mathbf{x} + b)$$

Define the margin of a linear classifier as the width that the boundary could be increased by before hitting a datapoint.

Support Vector Machines (SVM)





$$f(\mathbf{x}, \mathbf{w}, b) = sign(\mathbf{w}\,\mathbf{x} + b)$$

The maximum margin linear classifier is the linear classifier with the *maximum margin(!)*

This is the simplest kind of SVM (Called an LSVM – Linear SVM)

Support Vector Machines (SVM)

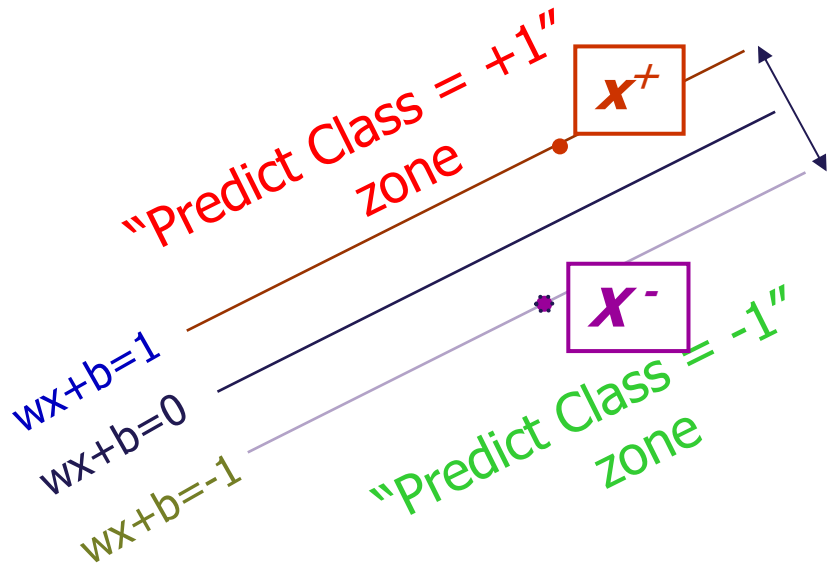
Why maximum margin?

1. Intuitively this feels safest.
2. If we've made a small error in the location of the boundary this gives us least chance of causing a misclassification.
3. Implies that only support vectors are important; other training examples are ignorable.
4. Empirically it works very well.

How do we represent this mathematically?

...in *m* input dimensions?

Support Vector Machines (SVM)



What we know:

- $\boldsymbol{w} \cdot \boldsymbol{x}^+ + b = +1$
- $\boldsymbol{w} \cdot \boldsymbol{x}^- + b = -1$
- $\boldsymbol{w} \cdot (\boldsymbol{x}^+ - \boldsymbol{x}^-) = 2$

$$M = \frac{(x^+ - x^-) \cdot w}{|w|} = \frac{2}{|w|}$$

Support Vector Machines (SVM)

Goal:

1. Correctly classify all training data
   $wx_i + b \geq 1$ if $y_i = +1$
   $wx_i + b \leq -1$ if $y_i = -1$
   $y_i(wx_i + b) \geq 1$ for all i

2. Maximize the margin $M = \frac{2}{|w|}$
   Same as minimize $\frac{1}{2}w^t w$

Solve with Quadratic Programming:

Minimize $\Phi(w) = \frac{1}{2}w^t w$
subject to $y_i(wx_i + b) \geq 1 \ \forall i$

Support Vector Machines (SVM)

Hard margin: So far we require all data points to be classified correctly

- No training error

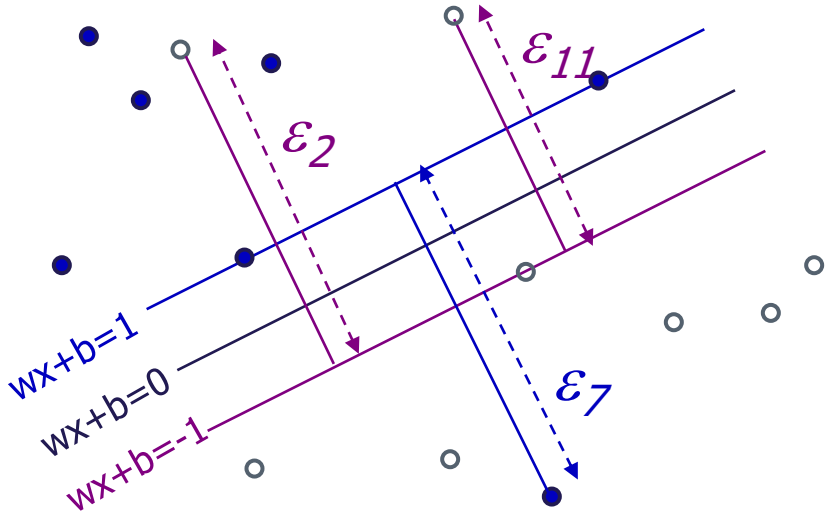
What if training set is noisy?

- Non-linear solution – could be overfitting!

Soft margin: Introduce slack variables

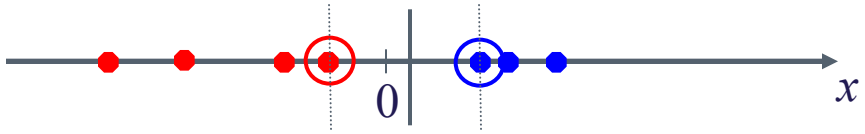
Minimize $\Phi(w) = \frac{1}{2} w^t w + C \sum_{k=1}^{R} \varepsilon_k$

subject to $y_i(wx_i + b) \geq 1 - \varepsilon_i,\ \varepsilon_1 \geq 0\ for\ all\ i$

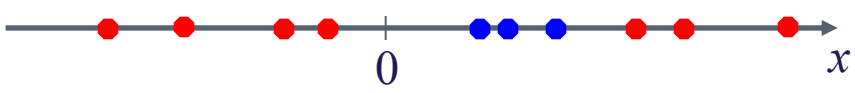


Non-linear SVMs

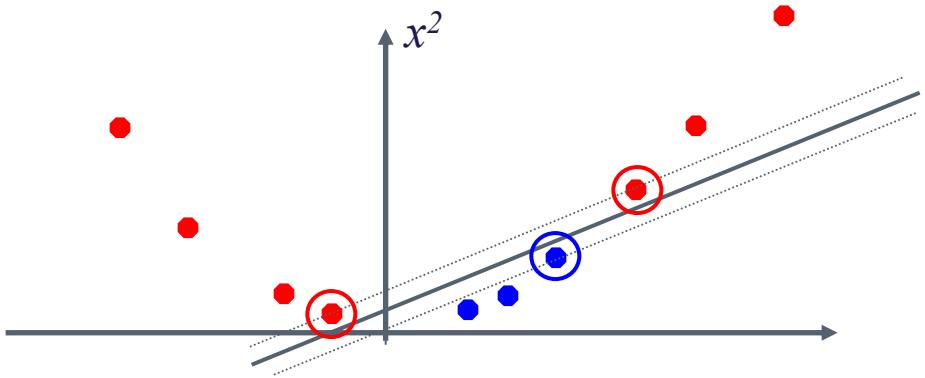
Datasets that are linearly separable with some noise work out great:



But what if the dataset is just too hard?

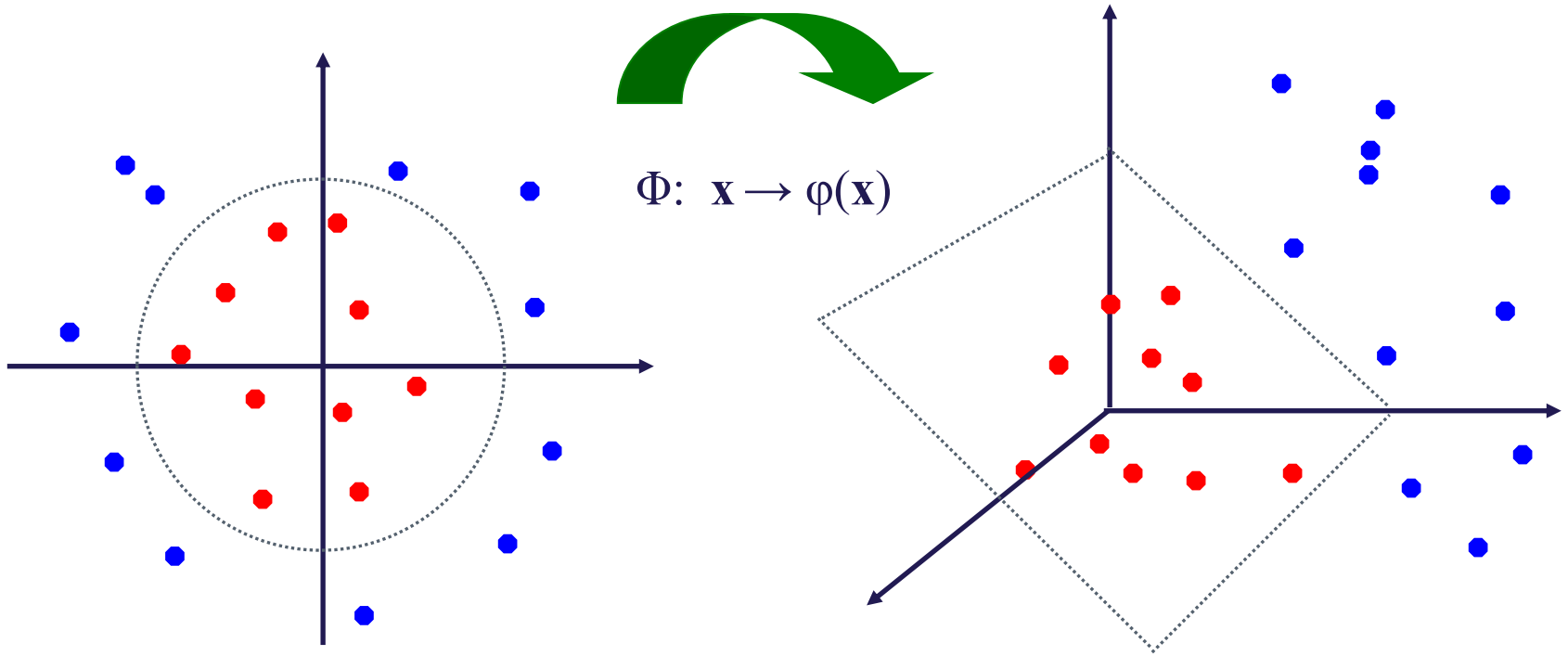


How about… mapping data to a higher-dimensional space:



Non-linear SVMs

- Idea: the original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



Use Kernel functions…

Support Vector Machines (SVM)

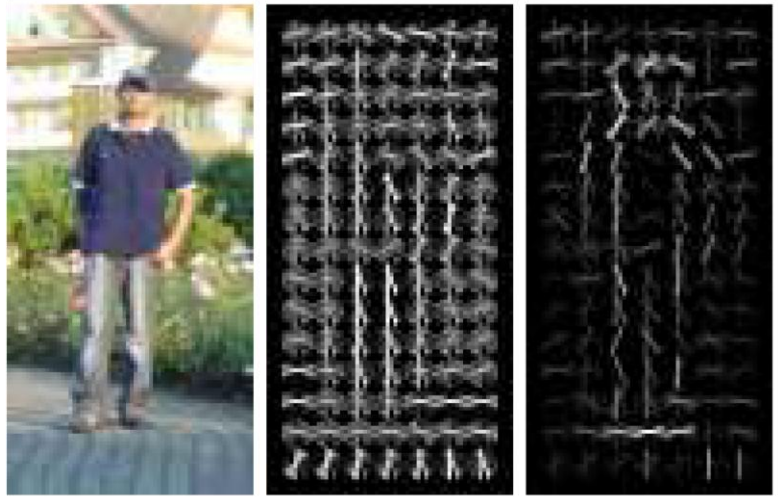
Properties of SVM

- Sparseness of solution when dealing with large data sets
  - Only support vectors are used to specify the separating hyperplane
- Ability to handle large feature spaces
  - Complexity does not depend on the dimensionality of the feature space
- Overfitting can be controlled by soft margin approach
- Nice math property: a simple convex optimization problem which is guaranteed to converge to a single global solution
- Not designed for multi-class classifications
  - Can be solved by combining several binary classifiers

SVM application

Dalal and Triggs ‘05:
Train SVM on HOG features of image
2 classes, person/not person

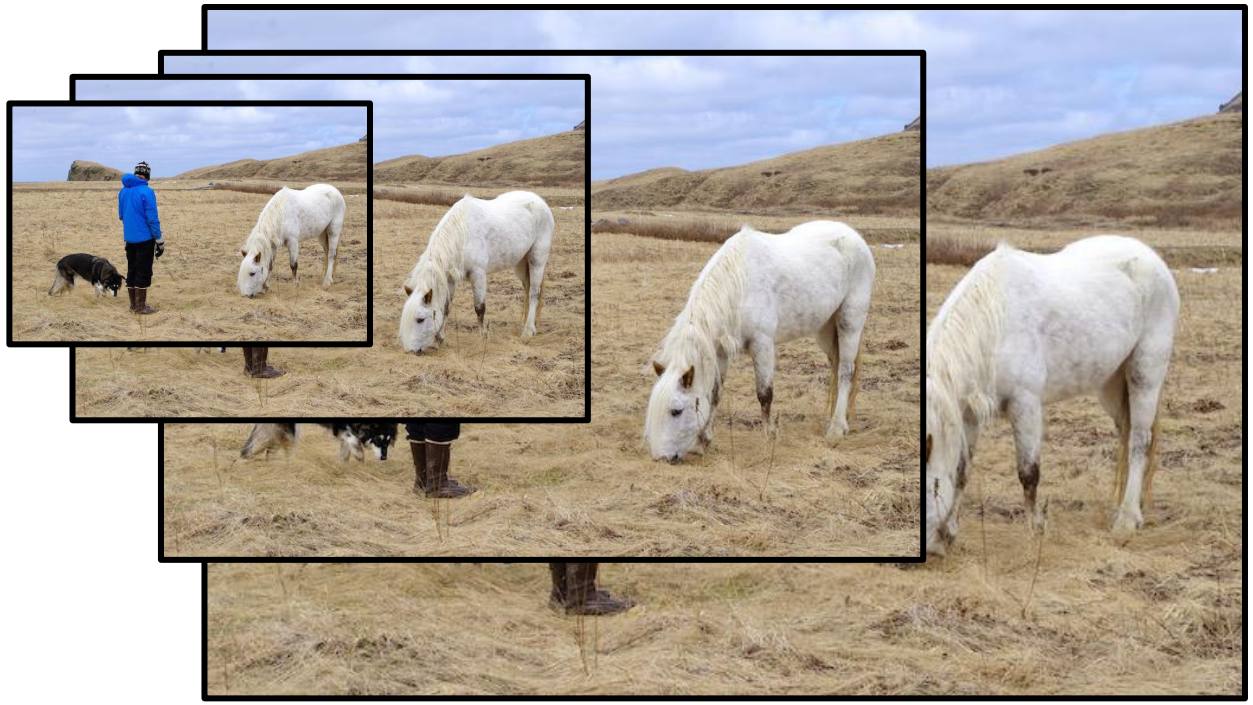
At test time:
Extract HOG features at many scales
Run SVM classifier at every location
High responses = person?



SVM application

Dalal and Triggs is a sliding window detector

- Many scales
- Every location



SVM application

Dalal and Triggs is a sliding window detector

- Many scales
- Every location

… Many trials later



Machine learning

In general, many of these ”classic” methods perform very well

But, you need to pick the right features to feed into the classifier!

- What are the right features in a given problem?

Possible solution: let the algorithm decide!

- Neural networks!

A note on parameter fitting and evaluation

- Choose parameters that provide best results on your entire dataset
  - Bad – overfitting to training data



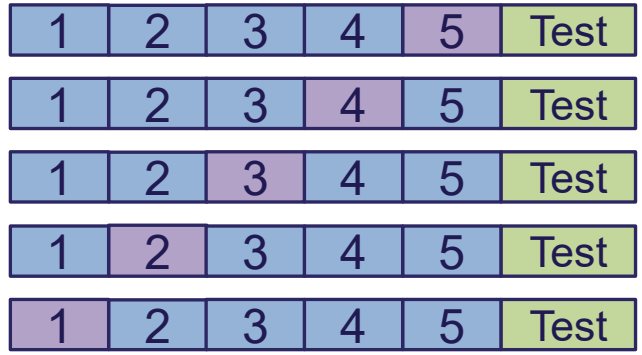
- Split into train and test sets, choose parameters based on results on test set
  - No idea how algorithm will perform on new data?



- Training, validation, and test. Choose parameters based on validation. Evaluate on test set.
  - Better!



- Cross-validation. Split data into folds, try each fold as validation and average the results
  - Very good for small datasets
  - Not really used for neural networks



Exercises

- Tutorial/exercises available on Moodle
- Question: How do you prefer the TA to help you?

